TEL354 · REDES DEFINIDAS POR SOFTWARE · 2026-2 · GRUPO 3 · H0892

Biblia técnica de la arquitectura SDN

Solución de seguridad para una red de campus académico. Documento interno de referencia: el trasfondo, paso a paso, de todo lo que la presentación parcial v2 muestra en una lámina.

USO INTERNO · NO DISTRIBUIR | versión 1.0 · octubre 2026 | alineada con la presentación v2

Rol	Integrante
Líder	Rodrigo Franco Rojas Soplin
Arquitecto	Piero Oropeza León
PM / QA	Camila Altamirano Ramírez
DevOps / SRE	Fernando Vincenzo Tavella Rivero
Coach	Germán Espinoza
Evaluador	Prof. Christian Quispe Ordoñez

Base documental: repositorio del grupo (commit 9626b2b, fases A–H, flujos 00–05, acceso 00–02) + observaciones del coach. Donde esta biblia amplía o corrige al repositorio, se indica en el capítulo 13.

Contenido

0 **Cómo usar este documento**
mapa presentación ↔ biblia, convenciones, estado de cada requisito

1 **Contexto, alcance y principios**
qué resolvemos, P1–P13, estilos y patrones

2 **La arquitectura, capa por capa**
planos, capas, 10 servicios y responsabilidades

3 **Interfaces I1–I9 y protocolos**
puertos, formatos, sincronía, errores

4 **Infraestructura v2**
topología, VLAN, direccionamiento, puertos, NIC, despliegue

5 **Plano de control**
diseño in-band, prototipo out-of-band, clúster ONOS, mastership

6 **Plano de datos**
pipeline, prioridades, cookies, meters, grupos, caminos, TCAM

7 **Identidad, sesiones y perfiles**
portal, RADIUS/LDAP, TOTP, cookies, ligadura, MAC_Moved

8 **Eventos, datos y persistencia**
envoltura, catálogo, colas, esquemas, retención

9 **R1 paso a paso**
18 pasos de la Fig. 10, con mensajes y fallos

10 **R2 paso a paso**
16 pasos de la Fig. 12, pipeline y camino denegado

11 **R4 paso a paso**
bucle cerrado, detección, escalera, presupuesto de latencia

12 **R3 y R5 (diseño), fallos y pruebas**
matriz de fallos, pruebas V1–V7, RD1–RD4, métricas

13 **Configuración, arranque, decisiones y abiertos**
parámetros, orden de arranque, ADR, discrepancias, glosario

0 Cómo usar este documento

La presentación (11 láminas) es para el público: dice qué y dónde. Esta biblia es interna: dice cómo, por qué y qué pasa cuando falla.

0.1 Dos documentos, un solo modelo

Ambos documentos describen **la misma arquitectura**. La presentación comprime; la biblia expande. Cada sección de la biblia cita las figuras de la presentación con una etiqueta azul como esta: Pres. Fig. 10 · lám. 7 (figura 10, lámina 7). Si algo no coincide entre ambos documentos, **manda la presentación v2** (es lo que el coach revisó) y la discrepancia debe anotarse en el capítulo 13.

0.2 Mapa presentación ↔ biblia

Lámina	Contenido	Figuras	Se amplía en
1	Portada, alcance, cambios v2	Fig. 1	cap. 1, 2
2	Arquitectura capas × planos	Fig. 2	cap. 2, 3
3	Módulos, interfaces y protocolos	Fig. 3, Tabla I1–I9	cap. 2, 3, 8
4	Infraestructura física y lógica v2	Fig. 5	cap. 4
5	Plano de control: in-band vs out-of-band	Fig. 6, Tabla 2	cap. 5
6	Despliegue y tolerancia a fallos	Fig. 8, 9	cap. 4, 5, 12
7	R1 — acceso según rol	Fig. 10, 11	cap. 7, 9
8	R2 — flujo de elevación	Fig. 12	cap. 10
9	R2 — clasificación, pipeline, TCAM	Fig. 13–16	cap. 6, 10
10	R4 — DDoS / brute-force	Fig. 17–19	cap. 11
11	R3, R5, trazabilidad, coach	Fig. 20, 21	cap. 12, 13

0.3 Estado de cada requisito

Req.	Peso	Estado	Qué significa en la práctica
General	30	Diseño + prototipo	Arquitectura, infraestructura, interfaces, despliegue (cap. 1–6, 13).
Doc/Pres.	10	Entregable	Presentación pública + esta biblia interna.
R1	10	**Implementado**	Acceso por rol: portal, RADIUS/LDAP, TOTP, perfiles BASE/ACADÉMICO/OPERADOR (cap. 7, 9).
R2	10	**Implementado**	Recursos privilegiados con elevación temporal y aprobación humana (cap. 10).
R4	20	**Implementado** · diferenciador	Detección adaptativa + escalera de mitigación + verificación (cap. 11).
R3	20	Solo diseño	IP spoofing, MAC_Moved, scanning: reutiliza R4 (cap. 12.1).
R5	20	Solo diseño	IDS en espejo de NC1/NC2 + bloqueo en A4 (cap. 12.2).

0.4 Convenciones

- **Direcciones**: gestión `10.0.0.0/24` (VLAN 20), usuarios `10.1.0.0/24` (VLAN 10), servidores `10.2.0.0/24` (VLAN 30), externa `10.3.0.0/24` (VLAN 40).

- **Prioridades OpenFlow**: número mayor = mayor autoridad (65000 canal de control in-band, 1000 mitigación, 200 sesión/elevación, …). Cap. 6.
- **Cookies**: `SES-…` sesión, `ELV-…` elevación, `INC-…` incidente. El controlador las traduce a 64 bits (cap. 6.4).
- **Valores de ejemplo** (MAC, IDs, tasas) son ilustrativos; los **valores iniciales** de configuración son los del cap. 13 y se calibran con medición (P12).
- **Verde / ámbar / naranja** en los cuadros: decidido · supuesto o por validar · cambio v2.

REGLA DE HONESTIDAD TÉCNICA

Esta biblia separa lo que **está decidido y escrito** en el repositorio de lo que es **supuesto de diseño** (por ejemplo, las IP .30 y .40 de los nodos ONOS adicionales, o la memoria de 512 MB por host académico). Los supuestos se marcan y se listan en el cap. 13.4. Nada de lo que se declare «medido» lo está hasta que el laboratorio lo cierre (V1–V7, M1–M4).

1 Contexto, alcance y principios

Qué problema resolvemos, bajo qué reglas de diseño y con qué estilos arquitectónicos.

1.1 El problema

Un campus académico mezcla poblaciones con necesidades opuestas: estudiantes y docentes (miles de dispositivos no administrados), operadores de TI y administración (poco numerosos y muy privilegiados), servidores de distinta criticidad y un borde con Internet. Una red tradicional resuelve esto con VLAN y ACL estáticas por puerto: lentas de cambiar, sin contexto, sin respuesta automática. La solución propuesta usa **SDN** para que **la política sea una decisión de software, central y auditable**, ejecutada a velocidad de línea por los switches.

El lema de la portada Pres. Fig. 1 · lám. 1 resume el reparto: **la plataforma decide; el controlador traduce; los switches ejecutan**. Todo el resto del documento es la consecuencia de tomar esa frase en serio.

1.2 Alcance del prototipo

- **Implementado**: R1, R2, R4 sobre topología física real de 10 switches Pica8 y 24 VMs (cap. 4).
- **Solo diseño**: R3 y R5, con la cadena de R4 como motor (cap. 12).
- **Fuera de alcance**: IPv6, VPN, WiFi/802.1X real en producción, feeds externos de inteligencia de amenazas (RP-06), alta disponibilidad de RabbitMQ/PostgreSQL (se documenta la referencia).
- **Simulado**: el IdP institucional (FreeRADIUS + OpenLDAP simulan lo que una universidad real ejecutaría; se simula la *instancia*, no el *protocolo*).

1.3 Los 13 principios (P1–P13)

Son las reglas contra las que se juzga cualquier decisión. Se citan por número a lo largo de toda la biblia.

#	Principio	Consecuencia concreta en el diseño
P1	Separación de planos	Nadie habla con los switches salvo ONOS (I1). Los servicios hablan con ONOS por I2. El Monitor observa vía ONOS, nunca directo.
P2	Decisión centralizada, ejecución distribuida	Una sola decisión (Políticas); la ejecución queda en el switch de ingreso. Si la plataforma cae, **lo instalado sigue funcionando** (arranque degradado).
P3	Responsabilidad única	Cada servicio responde una pregunta (Fig. 3). Una base por servicio, un usuario de BD por servicio.
P4	Complejidad justificada	Se añade un mecanismo solo si un requisito lo exige (p. ej. el clúster ONOS se justifica por la observación del coach).
P5	Mínimo privilegio, denegación por defecto	El perfil BASE solo alcanza DHCP, DNS, ARP y el portal. Todo lo demás se otorga con regla explícita.
P6	La identidad se exige solo para elevar	Presencia física = BASE sin credenciales; subir de peldaño exige más identidad (Fig. 11).
P7	La MAC es atributo, no credencial	Una MAC conocida no concede nada; la sesión liga identidad ↔ MAC ↔ puerto ↔ IP.
P8	El detector observa; la política decide; la red ejecuta	Detección clasifica, no ordena. Solo Políticas emite órdenes I2.
P9	Mitigar cerca del origen sin cortar tráfico legítimo	Meter/DROP en el switch de ingreso del atacante, no junto al destino; escalera proporcional.

P10	Todo privilegio y toda mitigación es temporal y reversible	TTL en toda orden; retiro por cookie; nunca se toca una regla ajena.
P11	Trazabilidad y observabilidad por diseño	Huecos declarados, no inventados; auditoría con cadena de hash; la verdad del plano de datos manda.
P12	Sin métrica no hay decisión válida	Umbrales configurables y calibrados por medición; «V» y «M» del cap. 12.
P13	Comunicación por eventos entre componentes de seguridad	El catálogo de eventos es el contrato; el evento informa, el comando ejecuta.

1.4 Estilos y patrones

Cuatro estilos, cada uno responde a una dimensión distinta **Pres. Fig. 2 · lám. 2**:

Estilo	Dimensión que resuelve	Dónde se ve
Capas	Estructura: quién puede depender de quién	Interacción → Aplicación → Soporte → Control → Infraestructura
Eventos / broker	Reacción desacoplada	RabbitMQ y el catálogo de 13+ eventos (cap. 8)
Cliente-servidor	Peticiones con respuesta	Portal, Consola, órdenes I2, consultas I7
Microservicios	Descomposición y despliegue independiente	10 servicios con base propia

Tres patrones: **Broker** (RabbitMQ entre servicios), **Pipe-and-Filter** solo dentro de Detección (captura → features → clasificación → alarma) y **Repository** (cada servicio es dueño de su persistencia; no hay JOIN entre bases).

> **LA REGLA QUE MÁS SE PREGUNTA: «EVENTO INFORMA, COMANDO EJECUTA»**
>
> Lo que **exige confirmación** (aplicar un perfil, instalar una mitigación) viaja **síncrono por REST (I2)**; su eco se publica por AMQP (I5) para Auditoría y Consola. Si una orden viajara por el bus, el emisor no sabría si la regla llegó al switch. Por eso `MitigationRequired` y `ElevationGranted` son eventos de *eco*: informan que ya ocurrió (o que se pidió), no lo provocan.

2 La arquitectura, capa por capa

Planos, capas y los diez servicios: qué hace cada uno, qué consume, qué publica, qué pasa si cae.

2.1 Planos × capas

La arquitectura se lee en dos ejes a la vez **Pres. Fig. 2 · lám. 2**. El eje vertical son los **tres planos SDN**; el horizontal son las **capas** lógicas de la solución.

Plano	Qué contiene	Qué lo caracteriza
Gestión	Los 10 servicios de la plataforma + soporte (RabbitMQ, PostgreSQL, FreeRADIUS, LDAP, chrony, DNS, Grafana)	Decide, registra y expone. Habla REST, AMQP, SQL, RADIUS. **Nunca** toca un switch.
Control	Clúster ONOS (3 nodos Atomix/Raft) + app G3	Traduce órdenes en OpenFlow, mantiene topología y estado de red, confirma con BARRIER. Único habitante de la frontera I1/I2.
Datos	10 switches Pica8/PicOS	Ejecuta a velocidad de línea. Reporta con eventos asíncronos (PACKET_IN, PORT_STATUS, FLOW_REMOVED) y contadores (MULTIPART).

Capa	Componentes	Pregunta que responde
Interacción	Portal `.5`, Consola `.17`, Grafana `.26`	¿Cómo entra una persona al sistema?
Aplicación	Identidad (Registro, IAM) · Observabilidad (Monitor, Detección, Adaptador IDS) · Respuesta (Incidentes, Políticas) · Gobierno (Auditoría)	¿Quién es? ¿Qué pasa? ¿Qué hacemos? ¿Quién lo hizo?
Soporte	RabbitMQ, PostgreSQL, FreeRADIUS, OpenLDAP, chrony, DNS, DHCP (ref.)	¿Con qué servicios básicos cuenta la aplicación?
Control SDN	Clúster ONOS 2.7 + app G3	¿Cómo se convierte una decisión en reglas?
Infraestructura	10 switches, enlaces, VLAN	¿Dónde se ejecuta?

2.2 Los diez servicios, uno por uno

Cada servicio está escrito en Python 3.12 (FastAPI + SQLAlchemy + pika). Todos exponen `/health` en dos niveles: *liveness* (el proceso vive) y *readiness* (su base y sus colas responden). Un servicio no *ready* no recibe tráfico, pero **la red no depende de él** (P2).

Servicio	IP	Protocolos	Responsabilidad	Publica / consume	Si cae
Portal	10.0.0.5	HTTPS 443 (I3)	Única superficie que el perfil BASE alcanza (regla p150). Muestra el selector de población, recibe credenciales, solicita elevación.	Reenvía a IAM por REST. No valida nada, no ordena a la red.	Sin portal no hay login nuevo; los dispositivos con sesión siguen.
IAM / AAA	10.0.0.10	REST · RADIUS 1812/1813 · AMQP	Autentica (RADIUS→LDAP para la comunidad; bcrypt+TOTP para operadores), crea sesiones, las liga a MAC/puerto/IP, invalida en `MAC_Moved`.	Pub: SessionOpened/Closed, ElevationRequested. Sub: MAC_Moved. BD: `bd_iam`.	Sin sesiones nuevas; las vigentes siguen hasta su timeout.

Registro	10.0.0.11	REST (I7)	Catálogo de dispositivos **privilegiados** (no de los 16 académicos). El alta es acto administrativo (P7).	Pub: DeviceRegistered/Revoked. Sub: DeviceConnected. BD: `bd_registro`.	No se pueden registrar ni verificar dispositivos privilegiados nuevos.
Monitor	10.0.0.12	REST cliente I8 · AMQP	Sondea contadores vía ONOS cada 5 s (caliente) / 60 s (barrido), mantiene EWMA por destino, **verifica** mitigaciones.	Pub: MetricSample, MitigationVerified, MAC_Moved. Sub: DeviceConnected, MitigationApplied. BD: `bd_monitor`.	Sin detecciones nuevas; las mitigaciones vigentes siguen (expiran por TTL).
Detección	10.0.0.13	AMQP	Pipe-and-Filter: recibe MetricSample, aplica umbral relativo con piso, N ventanas e histéresis; clasifica (flood, distribuido, brute-force, scanning). **Clasifica, no ordena** (P8).	Pub: AnomalyDetected. Sub: MetricSample. Lee umbrales de `bd_politicas` (vía parámetros).	Sin detecciones nuevas.
Incidentes	10.0.0.14	AMQP · REST (I7)	Máquina de estados del incidente (ABIERTO → EN_MITIGACIÓN → MITIGADO → CERRADO / ESCALADO). Ordena eventos por `secuencia`. Reconcilia contra I2 tras una caída.	Pub: IncidentOpened. Sub: AnomalyDetected, MitigationVerified/Expired, MAC_Moved. BD: `bd_incidentes`.	La cadena de respuesta espera; la red no.
Políticas	10.0.0.15	REST cliente I2 · AMQP	Evalúa ABAC y la escalera. **Único servicio que emite órdenes I2**. Retira por cookie.	Pub (eco): MitigationRequired, ElevationGranted/Expired. Sub: IncidentOpened, SessionOpened, ElevationRequested, MitigationVerified. BD: `bd_politicas`.	Sin órdenes nuevas; lo instalado sigue.
Auditoría	10.0.0.16	AMQP · SQL	Consume todo el catálogo (cola `auditoria` con binding `#`) y actos administrativos I7. Export JSONL con cadena SHA-256.	Sub: todo. BD: `bd_auditoria` (retención 90 d).	Huecos declarados en el bus; no se pierde evidencia ya escrita.
Consola	10.0.0.17	HTTPS 443 · REST I7	UI de operadores: alertas en vivo, aprobación de elevaciones y cuarentenas, edición de umbrales/perfiles. Agrega las API I7 de cada servicio.	Sub: AnomalyDetected, IncidentOpened, DeviceRegistered/Revoked. Sin base propia.	No hay aprobación humana (se bloquea QUARANTINE y elevaciones).
Adaptador IDS	10.0.0.18	EVE JSON → AMQP	Lee `eve.json` de Suricata, traduce alertas a `AnomalyDetected` con `origen=perimetro`.	Pub: AnomalyDetected.	R5 pierde detección; la red no se corta.

Pres. Fig. 3 · lám. 3 resume la misma tabla como tarjetas «una pregunta cada servicio».

2.3 Soporte de la plataforma

Componente	IP · puerto	Función	Notas de operación
RabbitMQ 3.13	10.0.0.21 · 5672	Exchange `plataforma` (topic, durable), colas por servicio + `inc.<id>`, DLQ por cola	Un usuario por servicio con permisos solo sobre sus colas.

PostgreSQL 16	10.0.0.22 · 5432	Una instancia, seis bases, un usuario por base sin grants cruzados	`pg_dump` diario; drill de restauración.
FreeRADIUS 3.2	10.0.0.23 · UDP 1812/1813	IdP simulado; cliente único = IAM	Accounting Start/Interim 300 s/Stop.
OpenLDAP 2.6	10.0.0.24 · TCP 389	Directorio `ou=comunidad`; bind de solo lectura	Cuentas de prueba, índice `uid`.
chrony 4.5	10.0.0.25 · UDP 123	Fuente de tiempo; todos los nodos clientes; UTC en todo registro	Sin tiempo fiable, los servicios que firman **esperan**.
Grafana 11	10.0.0.26 · TCP 3000	Tableros sobre `bd_monitor` y `bd_auditoria` (solo lectura)	Sin tableros, la observación cruda sigue.
Suricata 7	10.0.0.27 (gestión)	IDS en espejo (SPAN) de NC1/NC2; nunca inline	El puerto espejo no lleva IP.
DNS	10.0.0.28 · 53	Resolución para los hosts (regla p10 deja pasar DNS)	—

CAMBIO V2: DÓNDE VIVEN ESTOS SERVIDORES

En la v1 la plataforma colgaba de **A3** (VLAN 20 vía NIC-A). En la v2 **sube a la zona de control y servicios (C1/C2)**, colgada del núcleo, dual-homed. A3 queda libre y los servidores sensibles dejan de estar en el acceso. Ver cap. 4 y Pres. Fig. 5 · lám. 4.

2.4 Qué decide cada servicio y qué nunca decide

Decisión	Quién la toma	Quién NO puede
¿Esta persona es quien dice ser?	IAM (con IdP o TOTP)	Portal, Consola, ONOS
¿Qué perfil le corresponde ahora?	Políticas (ABAC)	IAM (solo identifica), Portal
¿Hay una anomalía?	Detección	Monitor (solo mide), Políticas
¿Qué mitigación aplicar?	Políticas (escalera)	Detección (P8), Incidentes
¿Cuarentena?	Un humano (aprobación en Consola)	Cualquier servicio automático
¿Qué reglas OpenFlow concretas?	ONOS (app G3)	Políticas (no dicta prioridades ni matches)
¿Funcionó la mitigación?	Monitor (contadores reales)	ONOS (solo confirma que instaló)

3 Interfaces I1–I9 y protocolos

Los contratos entre componentes: protocolo, puerto, formato, sincronía y qué hacer cuando fallan.

Toda interacción del sistema pasa por una de nueve interfaces. La tabla de la lámina 3 Pres. Fig. tabla I1–I9 · lám. 3 es el índice; aquí está el detalle.

ID	Entre	Protocolo · puerto	Sincronía	Autenticación
I1	ONOS ↔ switches	OpenFlow 1.3 · TCP 6653	mixta	whitelist de controladores en el switch
I2	Políticas / Monitor / IAM → ONOS	REST/HTTPS · JSON · TCP 8181	síncrona + BARRIER	clave de servicio por cliente
I3	Portal → IAM (y navegador → Portal)	HTTPS 443 → REST	síncrona	cookie de sesión opaca
I4	IAM → IdP	RADIUS UDP 1812/1813 → LDAP TCP 389	síncrona (3 s × 2)	secreto compartido RADIUS
I5	servicio ↔ servicio	AMQP 0-9-1 · TCP 5672	asíncrona	usuario por servicio
I6	servicio → su base	PostgreSQL · TCP 5432	síncrona local	usuario por base
I7	Consola → servicios	REST/HTTPS · JSON	síncrona	sesión del operador (SSO)
I8	Monitor → ONOS → switch	REST GET /contadores → OF MULTIPART	periódica 5 s / 60 s	igual que I2
I9 (v2)	ONOS ↔ ONOS	Atomix/Raft · TCP 5679 (mensajería) · 9876 (clúster)	asíncrona replicada	TLS / red de gestión

3.1 I1 — Southbound: OpenFlow 1.3

Aspecto	Valor	Por qué
Transporte	TCP 6653, desde el switch hacia el controlador (el switch inicia)	Los switches Pica8 configuran el endpoint de control; ONOS escucha.
Endpoints	Cada switch lleva **3 endpoints** (ONOS-1/2/3 · TCP 6653)	Permite que el switch se reconecte a otro nodo si cae el maestro.
Medio físico	**Prototipo**: interfaz `ma1` → red de gestión del laboratorio → 2.ª NIC de cada nodo ONOS. **Diseño**: in-band por VLAN 20 priorizada	Cap. 5.
Latido	`ECHO_REQUEST` cada 5 s; 3 sin respuesta ⇒ canal caído	Detección rápida de pérdida de control.
Confirmación	`BARRIER_REQUEST/REPLY` tras cada lote de instalación	Es lo que sostiene el estado «APLICADA» de I2.
Mensajes que usamos	HELLO, FEATURES_REQUEST/REPLY, ROLE_REQUEST/REPLY, FLOW_MOD, METER_MOD, GROUP_MOD, PACKET_IN, PACKET_OUT, PORT_STATUS, FLOW_REMOVED, MULTIPART_REQUEST/REPLY, BARRIER, ECHO	Subconjunto mínimo; nada que el switch no declare en FEATURES_REPLY (P1).
Negociación	El controlador **no puede ordenar más de lo que el switch declara**	Verificación V1.

Ciclo de vida de una conexión de control

1. El switch abre TCP 6653 hacia un controlador de su lista.
2. `OFPT_HELLO` en ambos sentidos; se negocia versión 1.3 (0x04).

3. ONOS envía `FEATURES_REQUEST`; el switch responde con datapath_id, nº de tablas, capacidades.
4. ONOS envía `ROLE_REQUEST` (MASTER o SLAVE/EQUAL) con `generation_id`; el switch rechaza roles con generación vieja.
5. ONOS instala el esqueleto de dispositivos conocidos y activa el descubrimiento LLDP (cap. 5.4).
6. Estado estable: `ECHO` cada 5 s; eventos asíncronos hacia el maestro.

3.2 I2 — Northbound: API de órdenes de ONOS

Es la **única puerta** para cambiar la red. Un solo recurso de órdenes y confirmación en dos niveles: **APLICADA** (ONOS recibió BARRIER_REPLY) y **VERIFICADA** (el Monitor vio caer el tráfico real; llega por el bus, nunca por esta API).

Recurso	Uso
`POST /api/v1/ordenes`	`APLICAR_PERFIL`, `APLICAR_MITIGACION`, `RETIRAR`
`GET /api/v1/ordenes?cookie=…`	Estado de una orden anterior (idempotencia, reconciliación)
`GET /api/v1/asociaciones?ip=…`	MAC ↔ switch:puerto ↔ IP vigente (ligadura de sesión)
`GET /api/v1/reglas?cookie=…&switch=…`	Reglas instaladas (reconciliación)
`GET /api/v1/topologia` · **`/contadores`**	Grafo de la red; contadores por puerto y flujo (I8)
`POST/GET /api/v1/servicios`	Inventario de servicios (anclajes del grafo)

```
POST /api/v1/ordenes   (HTTPS, TCP 8181, red de gestión)
{ "orden": "APLICAR_PERFIL",
  "dispositivo": {"mac":"52:54:00:01:01", "switch":"SA1", "puerto":"p3"},
  "perfil": "ACADEMICO",
  "vigencia": {"idle_timeout_s":1800, "hard_timeout_s":43200},
  "contexto": {"cookie":"SES-000123", "solicitante":"policy-engine",
               "motivo":"login-academico"} }
→ 201 {"estado":"APLICADA","reglas":["…"],"confirmada_en":"…"}
```

Campo	Regla
`contexto.cookie`	Clave de idempotencia: reaplicar la misma cookie produce las mismas reglas sin duplicados. Misma cookie con contenido distinto ⇒ **409**.
`vigencia`	Idéntica a la de la sesión: la regla y la sesión expiran juntas.
`contexto.solicitante`	Identidad del servicio que ordena; se audita. Un solicitante fuera de su competencia ⇒ **403**.
`APLICAR_MITIGACION.accion`	`RATE_LIMIT`, `BLOCK_SOURCE`, `ISOLATE_DEVICE`, `QUARANTINE`. Esta última exige `contexto.aprobacion` (id de aprobación humana) o ONOS la rechaza.
`RETIRAR`	Siempre **por cookie**; nunca se toca una regla ajena (P10).

Errores: 400 esquema inválido · 401 sin clave · 403 fuera de competencia · 404 dispositivo/cookie desconocido · 409 duplicada con cookie distinta · 503 controlador no disponible (el solicitante reintenta con retroceso; v2: cualquier nodo vivo del clúster atiende).

CAMBIO V2: I2 CON EL CLÚSTER

Cualquier nodo ONOS atiende I2. Si el nodo que recibe la orden no es el maestro del switch destino, el estado se coordina por Atomix y el maestro emite el FLOW_MOD. El cliente debería usar una lista de los tres nodos (`10.0.0.20/.30/.40:8181`) con reintento al siguiente ante `503` o timeout. *Pendiente de medir (M4): latencia de I2 por nodo.*

3.3 I3 e I7 — HTTP de personas y de operadores

- **I3** es lo único que ven las poblaciones: `GET /login` (selector «Comunidad universitaria» / «Personal de operación»), `POST /login`, `POST /logout`, `GET/POST /elevacion`. El selector solo elige la ruta de verificación; no concede nada.
- **I7** es la administración de cada servicio; la Consola es su cliente. La sesión del operador es el SSO. Respuesta `{datos, error:{codigo,mensaje}}`, paginación por cursor, `401` sin sesión, `403` sin rol.
- Rol mínimo por recurso: IAM/TI · Registro/TI-Admin · Incidentes/Admin-SA (aprobar cuarentena) · Políticas/TI-Admin · Auditoría/Auditor · Monitor/Admin-SA.

3.4 I4 — Identidad: RADIUS y LDAP

```
IAM ──Access-Request(User-Name, User-Password, NAS-IP)──► FreeRADIUS :1812/udp
FreeRADIUS ──bind + search (uid=…)──► OpenLDAP :389/tcp
FreeRADIUS ──Access-Accept(Class=perfil) | Access-Reject──► IAM
IAM ──Accounting-Request(Start, Acct-Session-Id=SES-…)──► :1813/udp
        … Interim cada 300 s … Stop al cerrar sesión
```

Timeouts: 3 s por intento, 2 reintentos; si falla, el login falla (el IdP es síncrono, la persona espera). Las credenciales de la comunidad **nunca** se almacenan en la plataforma. El reino de operadores no usa I4: vive en `bd_iam` con bcrypt + TOTP.

3.5 I5 y I6 — Bus y persistencia

Detalle completo del bus (envoltura, catálogo, colas, reintentos, DLQ) y de las bases en el cap. 8.

3.6 I8 — Observación: sondeo de contadores

Aspecto	Valor
Camino	Monitor → `GET /api/v1/contadores` → ONOS → `OFPMP_FLOW/PORT_STATS` → switch
Conjunto caliente	cada **5 s**: destinos protegidos y sus puertos de ingreso
Barrido completo	cada **60 s**: todos los flujos y puertos
Lote	una consulta por switch (10), no una por flujo — es lo que mide RNF-03
Tolerancia	si ONOS no responde en 2 s: **hueco de observación**; la línea base no se inventa (P11)
v2	el sondeo puede caer en cualquier nodo; cada maestro lee sus switches

3.7 I9 — Clúster ONOS (nueva en v2)

Es interna al plano de control. ONOS usa **Atomix** como almacén distribuido con consenso Raft: 3 nodos ⇒ quórum 2 ($\lfloor (n-1)/2 \rfloor = 1$ fallo tolerado). Lo que se replica: elección de maestro por dispositivo, tabla de cookies string↔64 bits, registro de asociaciones MAC↔puerto↔IP, inventario de servicios, órdenes en curso. Puertos: `TCP 5679` (mensajería ONOS entre nodos) y `TCP 9876` (clúster Atomix). Ver cap. 5.5.

3.8 Qué no es una interfaz (prohibiciones)

- Ningún servicio habla con un switch directamente (solo ONOS por I1).
- La Consola no toca bases de datos; usa I7.
- La Detección no ordena al controlador (P8).
- Ningún servicio lee la base de otro (Repository).

- Credenciales nunca viajan por el bus, ni en payload ni en correlación.

4 Infraestructura v2

Topología física, VLAN, direccionamiento, puertos, NIC y reparto de VMs/contenedores del servidor de laboratorio.

4.1 Topología física: 10 switches, 18 enlaces

La figura de referencia es **Pres. Fig. 5 · lám. 4**. Cuatro capas: **zona de control y servicios** (C1, C2), **núcleo** (NC1, NC2), **distribución** (D1, D2) y **acceso** (A1–A4). El principio rector es que **ningún fallo único de enlace o switch aísla un acceso ni un servidor**.

Grupo de enlaces	Cantidad	Detalle
Acceso → distribución	8	A1, A2, A3, A4 conectados cada uno a D1 y D2 (dual-homed)
Distribución → núcleo	4	D1 y D2 conectados cada uno a NC1 y NC2
Núcleo ↔ núcleo	1	NC1 ↔ NC2
Zona de servicios → núcleo	4	C1 y C2, cada uno a NC1 (directo) y NC2 (cruzado)
Zona de servicios ↔ zona	1	C1 ↔ C2
Total	**18**	

> **A CONFIRMAR CONTRA EL PLANO PUERTO A PUERTO**
> El desglose de las 18 aristas se reconstruye de la Fig. 5 (acceso dual-homed, C1/C2 «directo y cruzado» a NC1/NC2, NC1↔NC2). La asignación puerto a puerto (`05_puertos.md` del repositorio) todavía describe la v1 de 8 switches y debe actualizarse con la v2 (pendiente del arquitecto).

4.2 Papel de cada switch

Switch	Capa	Papel	Reglas dominantes
C1, C2	Control y servicios	Cuelgan plataforma, ONOS, servidores .10/.11/.12 e IDS. Segundo punto de control (defensa en profundidad).	p100 DROP a gestión desde VLAN 30/10; caminos p20 hacia servidores; p65000 canal (diseño in-band)
NC1, NC2	Núcleo	Transporte rápido; origen del espejo SPAN hacia Suricata.	caminos p20; sin política de usuario
D1, D2	Distribución	Agregan los 4 accesos; camino alternativo en falla.	caminos p20
A1, A2	Acceso	Usuarios: política de ingreso completa (esqueleto, sesión, mitigación).	todo el pipeline (cap. 6)
A3	Acceso	**Libre** en v2 (antes servidores y gestión). Solo esqueleto.	esqueleto
A4	Borde	Entrada externa (VLAN 40), origen de escenarios R5; bloqueo perimetral.	esqueleto + mitigación perimetral

4.3 VLAN y direccionamiento

VLAN	Nombre	Subred	Quién vive aquí
10	Usuarios	10.1.0.0/24	16 académicos `.10–.25` · operadores `.30–.31` · atacante interno `.50`

20	Gestión	10.0.0.0/24	Plataforma `.5` `.10–.18` · ONOS `.20` `.30` `.40` · RabbitMQ `.21` · PostgreSQL `.22` · RADIUS `.23` · LDAP `.24` · chrony `.25` · Grafana `.26` · Suricata `.27` · DNS `.28`
30	Servidores	10.2.0.0/24	general `.10` · privilegiado `.11` · crítico `.12` (BD institucional)
40	Externa	10.3.0.0/24	atacante externo `10.3.0.10` (origen R5)

DIRECCIONES .30 Y .40 DE ONOS

El repositorio fija solo ONOS-1 en `10.0.0.20`. Las direcciones `.30` y `.40` de ONOS-2/3 son **supuesto de la v2** (coinciden con la presentación). Confirmar que no choquen con ningún otro uso (p. ej. `.30` en VLAN 10 es un operador, pero en otra subred).

La VLAN no concede permisos: agrupa y direcciona; el permiso vive en las reglas del pipeline. **VLAN 20 ≠ canal OOB**: en el prototipo la VLAN 20 es un segmento del plano de datos y `ma1` es el canal de control; en el diseño in-band, la VLAN 20 también transporta OpenFlow (cap. 5).

4.4 Servidor de laboratorio: NIC y reparto

Un servidor KVM/libvirt de 32 GB con cinco NIC; cada NIC representa una población que entra por un puerto distinto de la red **Pres. Fig. 8 · lám. 6**.

NIC	Conecta a	Transporta	Población
NIC-A	Zona C1/C2 (trunk)	VLAN 20 + VLAN 30	Plataforma, ONOS, servidores (v2: antes A3 p6)
NIC-B	A1 p20	VLAN 10	16 hosts académicos (VM)
NIC-C	A4 p4	VLAN 40	Atacante externo
NIC-D	Espejo	— (sin IP)	VM del sensor Suricata (espejo SPAN de NC1/NC2 en v2)
NIC-E	A2 p6	VLAN 10	Operadores, atacante interno, puente de la demo MAC_Moved

Criterio VM vs contenedor: lo que la red debe ver como *equipo* (MAC, IP, puerto propios) es VM; un proceso sin identidad de red es contenedor.

Tipo	Elementos	Memoria
VM	16 académicos × 512 MB	8 GB
VM	2 operadores + 2 atacantes + 3 servidores × 1 GB	7 GB
VM	1 sensor Suricata	2 GB
Contenedores	10 servicios + RabbitMQ + PostgreSQL + FreeRADIUS + LDAP + chrony + DNS + Grafana + ONOS-1 (2 GB)	≈ 8 GB
Contenedores (v2)	ONOS-2 y ONOS-3 (2 GB c/u)	≈ 4 GB
Total estimado	17 GB de VM + 12 GB de contenedores	**≈ 29 GB de 32 GB**

MARGEN ESTRECHO

≈ 3 GB de holgura. Los hosts académicos bajaron de 1 GB a **512 MB** (imagen mínima) para que el clúster de 3 nodos quepa (supuesto de la v2; validar con la imagen real). Si falta memoria: (a) reducir a 12 académicos activos en las pruebas de R4, (b) mover Grafana fuera del servidor, (c) apagar el sensor fuera de las pruebas de R5.

LÍMITE HONESTO DEL CLÚSTER EN EL PROTOTIPO

Los 3 nodos ONOS comparten **un único servidor físico**. Toleran la caída de un *proceso o contenedor*, no la del host. En un campus real irían en tres hosts distintos (lám. 6, tabla «Qué se despliega y qué se diseña»). Es un límite del laboratorio, no de la arquitectura.

4.5 Redundancia de la capa de datos

Falla	Efecto	Mecanismo
Enlace A1–D1	A1 sigue por D2	PORT_STATUS → ONOS recalcula caminos, o fast-failover group si V3 lo permite
Switch D1	Los 4 accesos siguen por D2	Dual-homing
Switch NC1	Todo el núcleo por NC2	C1 sigue por su enlace cruzado a NC2
Switch C1	Servidores, plataforma y ONOS siguen por C2	Dual-homed a C1 y C2
Enlace NC1–NC2	Sin efecto visible	D1/D2 y C1/C2 ya tienen enlace propio a ambos

5 Plano de control

Dónde viaja OpenFlow (in-band en el diseño, out-of-band en el prototipo) y cómo un clúster de tres nodos ONOS sostiene la decisión.

5.1 La observación del coach, traducida

Observación	Interpretación adoptada	Consecuencia
Diseño in-band, implementación out-of-band	El **diseño de referencia** transporta OpenFlow por los enlaces de datos (VLAN 20 priorizada); el **prototipo** lo transporta por `ma1`	Misma interfaz l1; solo cambia el medio (Pres. Fig. 6 · lám. 5)
Redundancia en el plano de control: «dos clúster ONOS»	Un **clúster de 3 nodos** (no dos instancias independientes)	Raft exige quórum: con 2 nodos, la caída de uno deja sin quórum; con 3 se tolera 1 fallo

> **INTERPRETACIÓN A CONFIRMAR CON EL COACH**
>
> Si el coach quiso decir literalmente *dos clústeres* (p. ej. dos sitios activo/pasivo), cambia el diseño: habría que definir réplica entre clústeres. Hemos elegido **1 clúster × 3 nodos** por ser la forma estándar de ONOS y la que da quórum. Dejar constancia en la próxima reunión.

5.2 Diseño in-band

La IP de gestión de cada switch vive en una interfaz de la VLAN 20; OpenFlow viaja sobre los mismos enlaces que los datos, aislado y priorizado.

- **Regla de canal** (`p65000`, permanente) en cada tramo del camino switch↔C1/C2: `tcp_dst=6653 v tcp_src=6653` con `ip ∈ {ONOS-1,2,3}` → reenvío hacia/desde la zona de control. Ida y vuelta + LLDP ≈ **+4 reglas por switch**.
- **Prioridad de servicio**: el tráfico de control se marca **DSCP CS6** y se encola en una **cola estricta**; el tráfico de usuario no puede desplazarlo.
- **Nunca table-miss**: el control nunca debe caer al PACKET_IN, o se forma el bucle PACKET_IN → ONOS → …
- **Riesgo central**: un DDoS que sature un enlace compartido puede degradar el control. Se mitiga con la cola estricta y con R4, que corta el ataque en el switch de ingreso *antes del núcleo*.

Bootstrap in-band: quién va primero

Un switch no puede aprender reglas de ONOS si todavía no alcanza a ONOS. El orden correcto empieza por el borde de la zona de control y avanza hacia fuera:

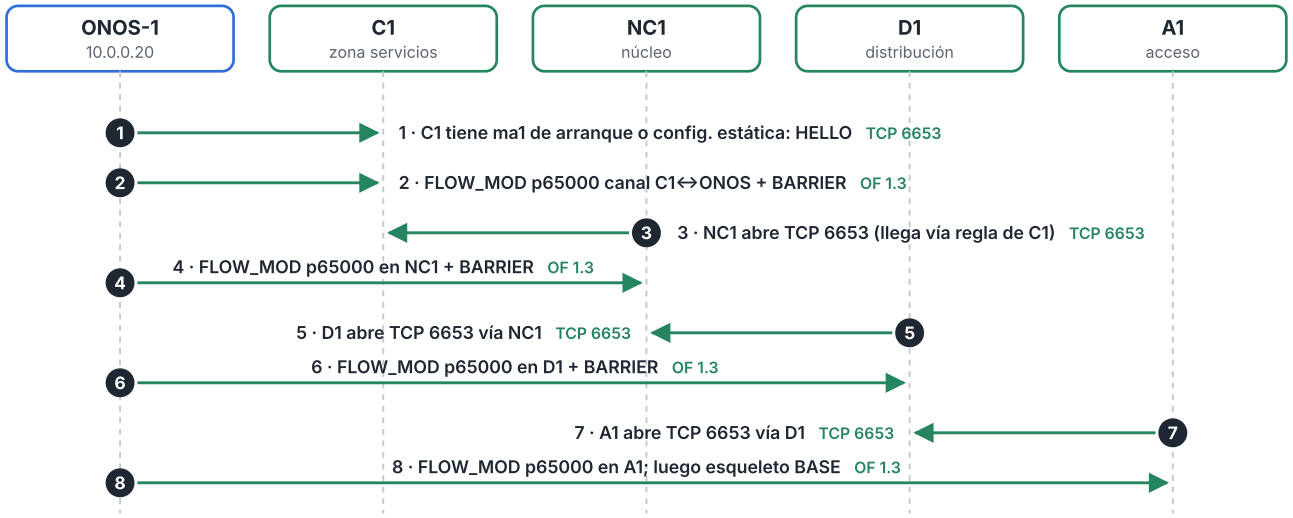


Fig. B-1 — Bootstrap in-band (diseño). Cada tramo habilita el canal del siguiente antes de instalar cualquier regla de usuario. El primer tramo (C1) se arranca con configuración estática, porque está pegado a los nodos ONOS. *Ilustra la lám. 5 (Fig. 6, izquierda).*

5.3 Implementación out-of-band (prototipo)

- Cada switch Pica8 tiene la interfaz de gestión `ma1` (fuera del plano de datos). Se configura con la IP de laboratorio y los tres endpoints `tcp:ONOS-n:6653`.
- La red de gestión del laboratorio es física y separada; cada nodo ONOS tiene una **2.ª NIC** en esa red (la 1.ª está en VLAN 20 para I2–I9).
- Arranque inmediato: el control no depende de reglas en el plano de datos. Las pruebas de R4 pueden saturar enlaces sin perder control.
- El switch acepta control solo de los endpoints en su whitelist; una conexión desde otra interfaz se rechaza (V6).

Aspecto	In-band (diseño)	Out-of-band (prototipo)
Medio de I1	enlaces de datos, VLAN 20	ma1 + red de gestión del laboratorio
Arranque	bootstrap escalonado (Fig. B-1)	inmediato
Reglas extra	+4 por switch	0
Riesgo	DDoS satura el enlace del control	sin enlace compartido
Contención	cola estricta + R4 en ingreso	—
Coste en campus	bajo (sin cableado extra)	alto fuera del laboratorio
Arquitectura	sin cambios: mismas I1–I9, mismo pipeline	sin cambios

5.4 Descubrimiento de topología (LLDP)

1. ONOS envía periódicamente, por cada puerto de cada switch, un `PACKET_OUT` con una trama LLDP (EtherType `0x88CC`) que codifica *(dpid, puerto)*.
2. El switch vecino no tiene regla para LLDP en un puerto de infraestructura → la trama sube como `PACKET_IN` a su maestro.
3. ONOS correlaciona *(dpid origen, puerto origen) → (dpid destino, puerto destino)* y añade la arista al grafo.
4. Un enlace caído genera `PORT_STATUS` y/o expira el LLDP; ONOS elimina la arista y marca los caminos afectados para recalcular.

En el diseño in-band, el LLDP también necesita regla propia (parte de las +4) para no depender de table-miss.

5.5 El clúster ONOS de 3 nodos

Cada nodo ejecuta ONOS 2.7 + app G3 y un miembro **Atomix** (consenso Raft). Todos ven el mismo estado.

Concepto	Cómo funciona
Quórum	⌊(n−1)/2⌋ = 1 fallo tolerado con n = 3. Con 2 nodos vivos hay mayoría y el clúster sigue decidiendo.
Mastership	Para cada switch, ONOS elige **un nodo MASTER** mediante elección de líder en Atomix; los otros dos son SLAVE/EQUAL. Solo el maestro envía FLOW_MOD y recibe PACKET_IN para ese switch.
Reparto	Los 10 switches se reparten entre los 3 nodos (≈ 3–4 cada uno). Es así como un ataque distribuido se mitiga «en cada ingreso» (cada maestro instala en sus switches).
Qué se replica	cookies (string↔64 bits), asociaciones MAC↔puerto↔IP, inventario de servicios, órdenes en curso, sesiones/incidentes que la app G3 guarde en el almacén compartido
Qué NO se replica	la verdad de las reglas: está en el **switch**; ONOS la reconcilia leyéndola (P11)
Entrada	cualquier nodo atiende l2 (REST 8181). Si no es el maestro del switch destino, enruta internamente.

Qué pasa cuando cae un nodo

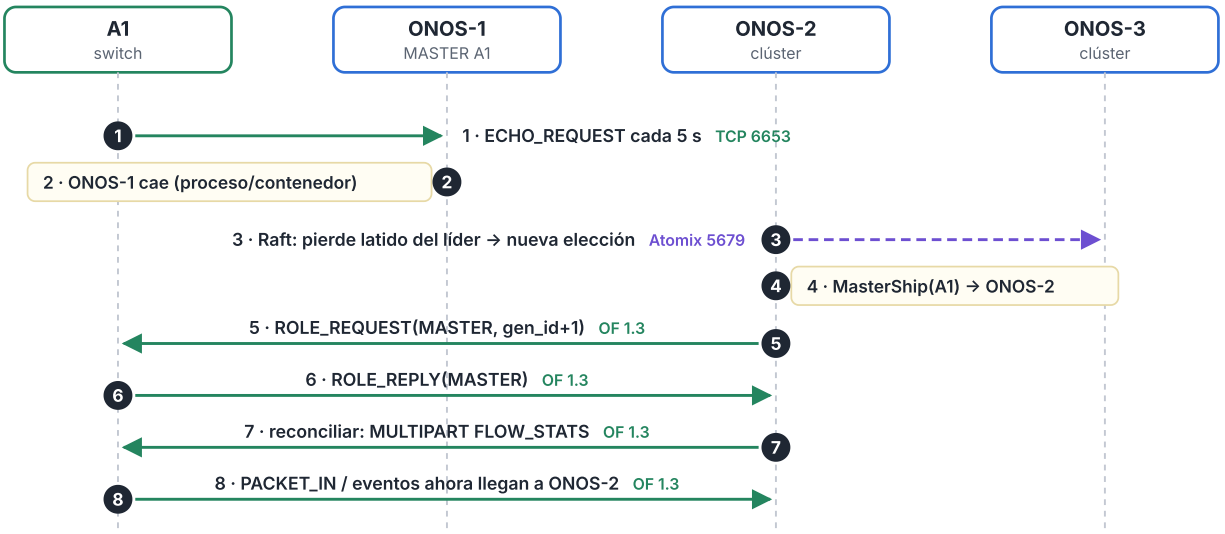


Fig. B-2 — Re-maestría tras la caída de ONOS-1. Las reglas ya instaladas en A1 **no se tocan**: el plano de datos sigue reenviando durante la conmutación. Solo las decisiones nuevas esperan. Mide M3 (tiempo de re-maestría). *Amplía la fila «Cae ONOS-1» de la Fig. 9, lám. 6.*

Escenario	Efecto en el plano de datos	Efecto en decisiones nuevas	Métrica
Cae 1 nodo	ninguno (reglas intactas)	pausa breve hasta re-maestría; luego continúan	tiempo de re-maestría (M3)
Caen 2 nodos	reglas instaladas siguen; expiran por timeout	**sin quórum**: no hay decisiones nuevas hasta recuperar uno	duración de degradación
Cae el enlace ONOS↔switch	el switch intenta los otros 2 endpoints	reasignación de maestro	tiempo de reconexión
Partición de red entre nodos	la mayoría (2) sigue; la minoría (1) pierde sus maestrías	la minoría queda inactiva	—

POR QUÉ LAS REGLAS SOBREVIVEN

Las reglas con `idle/hard_timeout` viven en el switch. Si el control desaparece, los flujos siguen hasta expirar (P2, P10). Por eso la caída del control **degrada la capacidad de decidir**, no la de reenviar.

6 Plano de datos

Cómo se materializa la política en los switches: pipeline, prioridades, cookies, meters, grupos, caminos y capacidad.

6.1 El principio: la prioridad es jerarquía de autoridad

Cada entrada del pipeline es una decisión de la plataforma convertida en *match + acción*. Cuando varias reglas coinciden con un paquete, gana la de **mayor prioridad**. Por eso el orden de autoridad es:

```
mitigación (1000) > sesión y elevación (200) > esqueleto BASE (150…100)
> caminos (20) > servicios básicos (10) > table-miss
(en diseño in-band, por encima de todo: canal de control p65000)
```

La mitigación manda sobre todo (es la respuesta al ataque); la sesión manda sobre el esqueleto (es el privilegio otorgado); el esqueleto manda sobre los caminos (es la línea base). **La política se aplica en el switch de ingreso** de cada flujo, no junto al destino Pres. Fig. 14 · lám. 9.

6.2 Pipeline lógico: dos etapas

Etapa	Pregunta	Prioridades	Dónde
Política	¿Este paquete puede pasar?	1000 · 200 · 150 · 120 · 110 · 100 · 10 · table-miss	switch de **ingreso** (A1–A4) y, como segundo control, C1/C2
Caminos	¿Por dónde sale?	20 (ipv4_dst → puerto) · 5 (inundación DHCP)	**todos** los switches del trayecto

Si la verificación **V1** confirma multi-tabla, cada etapa es una tabla física enlazada con `GOTO_TABLE`. Si no, ONOS compone una entrada única (match de política + salida): la especificación lógica no cambia, cambia la materialización. El cómo es responsabilidad del controlador.

6.3 Las prioridades, entrada a entrada

Prio.	Entrada	Match	Acción	Cookie	Vigencia
65000	Canal de control (solo in-band)	TCP 6653 ↔ ONOS · LLDP	reenviar	canal	permanente
1000	Mitigación RATE_LIMIT	ipv4_src (origen) → ipv4_dst (objetivo)	meter → caminos	INC-…	hard_timeout = ttl (300 s)
1000	BLOCK_SOURCE	origen → cualquiera	DROP	INC-…	ttl
1000	ISOLATE_DEVICE / QUARANTINE	dispositivo	DROP total	INC-…	10 min / TTL de la aprobación
200	Sesión (perfil)	dispositivo → alcance del perfil	→ caminos	SES-…	idle y hard de la sesión
200	Elevación	dispositivo → recurso elevado	→ caminos	ELV-…	hard_timeout = TTL (3600 s)
150	Portal pre-login	dispositivo → 10.0.0.5 (443/80)	→ caminos	esqueleto	permanente
120	Par aprendido	eth_src=MAC ∧ ip_src=IP	→ caminos	esqueleto	permanente
110	Anti-spoofing	ip_src=IP ∧ eth_src≠MAC	DROP	esqueleto	permanente
100	Denegación a gestión	dispositivo → 10.0.0.0/24	DROP	esqueleto	permanente

20	Caminos	ipv4_dst	→ puerto de salida	camino	mientras valga la topología
10	DHCP / DNS	UDP 67/68 · 53	→ caminos	esqueleto	permanente
—	Table-miss	—	PACKET_IN	—	—

Ver la tabla gráfica en **Pres. Fig. 14 · lám. 9**. **El esqueleto BASE** (150–10) es la línea base que todo dispositivo lleva antes de autenticarse: portal, par MAC/IP, anti-spoofing, denegación a gestión y DHCP/DNS (≈ 6 entradas).

> **POR QUÉ EL DROP A GESTIÓN (P100) ES PERMANENTE**
> La sesión (200) vence al esqueleto, pero **no lo reemplaza**: un académico con sesión sigue sin poder llegar a la VLAN 20. Solo una **elevación** con match específico y aprobación humana abre una excepción acotada y con TTL. Si el DROP fuera parte de la sesión, perder la sesión abriría la gestión: eso es exactamente lo que se evita.

6.4 Cookies: identidad de cada regla

La cookie OpenFlow es un campo de 64 bits que ONOS asigna a cada `FLOW_MOD`. Permite **retirar y auditar por dueño** sin tocar reglas ajenas (P10).

Bits altos	Tipo	Nombre lógico	Ejemplo
0x01	Sesión	`SES-xxxxxx`	`0x01…0123`
0x02	Incidente / mitigación	`INC-xxxx`	`0xC0000007` ↔ INC-0007 (en la presentación)
0x03	Esqueleto	—	—
0x04	Camino	—	—
0xE1...	Elevación (convención de la presentación)	`ELV-xxxx`	`0xE1000042` ↔ ELV-0042

> **DISCREPANCIA MENOR DE CODIFICACIÓN**
> El repositorio (G-06 §3) define tipos por los bits altos `0x01–0x04`; los ejemplos de la lámina 9 usan `0xE1…` y `0xC0…` para elevación e incidente. Son *valores de ejemplo* de la presentación. El registro de cookies de ONOS (tabla string↔64 bits persistida y replicada) es la fuente de verdad; hay que alinear la convención definitiva antes de la demo.

Operaciones por cookie: `RETIRAR` (FLOW_MOD DELETE con cookie + máscara), `GET /reglas?cookie=` (FLOW_STATS filtrado), y reconciliación tras una caída (comparar cookies vivas contra las registradas).

6.5 Meters (limitación de tasa)

- Un **meter por par (origen, objetivo)** limitado; banda de tipo `DROP` a la tasa del incidente.
- Dos unidades posibles en OpenFlow 1.3: `KBPS` (la presentación usa 35 Mbps = 35000 kbps) o `PKTPS` (el repositorio ejemplifica 1000 pps). **Lo decide V2** según lo que soporte el Pica8.
- ONOS lleva el inventario de meter_id y los libera al retirar la mitigación.
- Si V2 no confirma meters nativos: **degradación documentada** — contención por reglas temporales (bloquear flujos nuevos del origen mientras persiste el exceso).

```
OFPT_METER_MOD   command=ADD meter_id=7 flags=KBPS
   band: type=DROP rate=35000
OFPT_FLOW_MOD    command=ADD table=0 priority=1000 cookie=0xC0000007 hard=300
   match: in_port=p6 ipv4_src=10.1.0.50 ipv4_dst=10.2.0.10
   instr: meter:7 → apply_actions → output:p1
```

6.6 Caminos y redundancia (enrutamiento)

1. Los hosts no conocen la topología; ONOS construye el **grafo** con LLDP (cap. 5.4) y ancla los servicios declarados (`POST /servicios`).
2. Para cada destino ancla (p. ej. 10.2.0.11), calcula el camino de menor costo (**1 por salto**, con penalización manual configurable por enlace).
3. Instala una regla `p20 ipv4_dst → puerto` en **cada switch del camino** (o un grupo fast-failover por destino).
4. Con dual-homing, cada destino tiene **dos caminos disjuntos**. Si cae un enlace, PORT_STATUS → ONOS recalcula y reinstala; con grupos `FAST_FAILOVER` el propio switch conmuta sin esperar a ONOS.
5. El cálculo se rehace en cada cambio del grafo (caída/alta de enlace, alta de servicio).



Fig. B-3 — Recorrido de un paquete autorizado A1 → srv .11 (R2, tras elevación). Una sola regla de política (en el ingreso) y reglas de camino en el resto. *Complementa la Fig. 12, paso 13 (lám. 8).*

6.7 Capacidad de tabla (TCAM)

El presupuesto de reglas es un riesgo real en hardware: **Pres. Fig. 16 · lám. 9** estima entradas por switch en el peor caso concurrente (prototipo OOB), contra una capacidad supuesta de ≈ 2.000.

Switch	Entradas (peor caso)	Composición
A1, A2	188	16 base + 144 sesión + 8 caminos + 20 mitigación (A1 = 9,4 % de 2.000)
A4	35	esqueleto + mitigación perimetral
A3	22	solo esqueleto (libre en v2)
D1, D2, NC1, NC2	36	caminos hacia todos los destinos
C1, C2	44	caminos + p100 DROP a gestión (segundo control)

SUPUESTO A CERRAR CON V1

La capacidad de 2.000 es **supuesta**. V1 la mide con *llenado por lotes hasta TABLE_FULL* (RNF-05). En el diseño in-band se suman +4 reglas por switch (despreciable). Si la capacidad real fuera ≤ 500, A1/A2 pasarían a ser el cuello de botella.

6.8 Eventos que el switch envía y cómo se usan

Mensaje OF	Cuándo	Quién lo consume	Para qué

PACKET_IN	table-miss (primer paquete) o LLDP	ONOS (maestro)	aprender host, redirigir al portal, descubrir topología
PORT_STATUS	enlace sube/baja	ONOS	recalcular caminos; MAC_Moved si un host aparece en otro puerto
FLOW_REMOVED	regla expira o se borra (flag SEND_FLOW_REM)	ONOS → app G3	avisar expiración (ElevationExpired, SessionClosed)
MULTIPART_REPLY	respuesta a FLOW/PORT/METER stats	ONOS → I8	contadores para el Monitor
BARRIER_REPLY	tras un lote	ONOS	confirmar APLICADA
ECHO_REPLY	cada 5 s	ONOS	latido del canal

7 Identidad, sesiones y perfiles

Cómo se prueba quién es alguien, cómo se liga esa identidad a un dispositivo y a un puerto, y qué la invalida.

7.1 Las tres poblaciones y sus caminos

Población	Prueba de identidad	Dónde viven las credenciales	Rol resultante
Comunidad (académicos, docentes)	Portal cautivo → RADIUS → LDAP	IdP institucional (FreeRADIUS + OpenLDAP); la plataforma **no** las almacena	ACADÉMICO
Operadores (TI, Admin, Superadmin, Auditor)	Usuario + clave bcrypt + **TOTP** (RFC 6238, ventana ±1)	`bd_iam.operadores` (hash bcrypt; secreto TOTP cifrado)	TI / Admin / SA / Auditor
Dispositivo sin credencial	Ninguna: BASE	—	BASE

> **DISCREPANCIA A RESOLVER CON EL EQUIPO**
> La memoria del proyecto registra un ADR «802.1X + RADIUS para todos los usuarios». El repositorio y la presentación v2 usan **portal cautivo + RADIUS** para la comunidad y reservan **802.1X para puertos sensibles** de un despliegue real (H-01 §..., G-04 §6). Esta biblia sigue al repositorio y a la presentación. Si el ADR 802.1X es el vigente, hay que actualizar lámina 7 y los capítulos 7 y 9.

7.2 El selector de población

El portal muestra «Comunidad universitaria» y «Personal de operación». El selector **no concede nada**: solo elige la ruta de verificación. Credenciales en la ruta equivocada fallan y se auditan. No existe variante de operador sin TOTP.

7.3 Ciclo de vida de una sesión

```
login(usuario, secreto, poblacion, ip_origen):
  si poblacion == "operacion":
      verificar_bcrypt(secreto, op.hash); verificar_totp(op.totp_secreto, codigo, ±1)
  si poblacion == "comunidad":
      r = radius.access_request(usuario, secreto)        # I4
      perfil = r.atributos["perfil"]; radius.accounting("Start")
  asociacion = i2.asociaciones(ip=ip_origen)             # MAC + switch:puerto + IP
  sesion = crear_sesion(usuario, perfil, asociacion, ttl[perfil])
  publicar(SessionOpened)                                # Políticas ordena el perfil
  emitir_cookie_opaca(sesion)    # HttpOnly · Secure · SameSite=Lax

validar_peticion(cookie, ip_origen):
  s = sesiones[hash(cookie)]     # la validez vive en el SERVIDOR
  exigir(s.estado == activa y s.ip == ip_origen y ahora < s.hard_timeout)
  s.ultima_actividad = ahora

al_evento(MAC_Moved) si mac ∈ sesiones activas:
  cerrar_sesion(motivo="mac_moved"); radius.accounting("Stop"); publicar(SessionClosed)
```

Ligadura de la sesión

La sesión no es solo «usuario X»: es la tupla **(usuario, rol, MAC, switch:puerto, IP)**. Ese conjunto es lo que (a) valida cada petición y (b) rompe `MAC_Moved`. Una cookie robada y usada desde otra IP se rechaza e invalida la

sesión (RNF-01).

7.4 Escalera de perfiles

Cada peldaño exige más identidad y dura menos **Pres. Fig. 11 · lám. 7**:

Perfil	Cómo se alcanza	Idle	Hard / TTL	Alcance
BASE	presencia física	—	permanente	DHCP, DNS, ARP, portal (p150)
ACADÉMICO	login contra IdP	30 min	12 h	servicios académicos + Internet; no gestión, no privilegiados
ELEVADO	solicitud + aprobación de administrador	—	1 h	un recurso privilegiado concreto (match por ipv4_dst)
OPERADOR	registro + credencial + TOTP	15 min	8 h	según rol TI / Admin / SA

Permiso efectivo = Rol ∧ Recurso ∧ Acción ∧ Contexto ∧ Vigencia ∧ Estado de seguridad (ABAC). Un dispositivo bajo mitigación (p1000) pierde efecto de su sesión aunque esta siga vigente (P9).

7.5 Registro y elevación de operadores

1. TI registra el dispositivo del operador (I7) → `bd_registro` + historial.
2. El IAM emite el secreto TOTP **una sola vez** (QR en la Consola).
3. El operador verifica un código; recién entonces el par (dispositivo, identidad) queda habilitado (`provision_totp.verificado_en`).
4. Cada login posterior exige clave + TOTP + que la MAC coincida con la registrada (la MAC es atributo, no credencial).

7.6 Máquina de estados de la sesión



Fig. B-4 — Estados de una sesión y qué los provoca. Toda transición hacia abajo retira las reglas por cookie y emite un evento.

8 Eventos, datos y persistencia

El bus asíncrono (envoltura, catálogo, colas, entrega) y las seis bases de datos.

8.1 Envoltura común de todo evento

```
{ "evento_id": "018f2a…",       // UUIDv7: único; clave de idempotencia
  "tipo": "AnomalyDetected",     // nombre del catálogo
  "emisor": "deteccion",
  "timestamp": "2026-10-05T14:03:22Z",   // UTC (chrony)
  "secuencia": 7,                // monótona por incidente
  "correlacion": {"incidente":"INC-0042","sesion":"SES-000123",
                  "dispositivo":"52:54:00:01:01"},
  "payload": { … } }
```

- **evento_id** hace idempotente al consumidor: guarda los ids ya procesados (ventana 7 días) y descarta duplicados.
- **secuencia** hace ordenable al incidente: el consumidor acepta `esperada+1`; un salto se declara como hueco, jamás se procesa fuera de sitio.
- **correlacion** une registros entre servicios sin cruzar bases.
- **Nunca** viajan por el bus: comandos (I2), preguntas (I7) ni credenciales.

8.2 Catálogo de eventos

Tipo	Payload	Productor → consumidores
DeviceConnected	mac, ip, switch, puerto	ONOS → Registro, Monitor, Auditoría
MAC_Moved	mac, ubicacion_anterior, ubicacion_nueva	Monitor → Incidentes, Políticas, Auditoría, **IAM**
MetricSample	destino, ventana, pps, bps, flujos_nuevos, fuentes_distintas	Monitor → Detección
AnomalyDetected	tipo, objetivo, origenes[], tasas, linea_base, desviacion, severidad_propuesta	Detección / Adaptador IDS → Incidentes, Consola, Auditoría
IncidentOpened	incidente, tipo, objetivo, severidad	Incidentes → Políticas, Consola, Auditoría
MitigationRequired	incidente, accion, objetivo, origenes, parametros, ttl	Políticas → eco al bus (el comando va por I2)
MitigationApplied	incidente, reglas[], switches	ONOS → Monitor, Incidentes, Auditoría
MitigationVerified	tasas_antes, tasas_despues, cumple_objetivo	Monitor → Incidentes, Políticas, Auditoría
MitigationExpired	incidente, regla, motivo	Monitor/ONOS → Incidentes, Políticas, Auditoría
ElevationRequested	solicitante, perfil_pedido, alcance, duracion	IAM → Políticas, Consola, Auditoría
ElevationGranted / Expired	solicitante, perfil, ttl / motivo	Políticas → eco (comando por I2)
SessionOpened / Closed	identidad, perfil_o_rol, dispositivo / motivo	IAM → Políticas, Auditoría
DeviceRegistered / Revoked	mac, titular, rol, responsable, vigencia	Registro → Auditoría, Consola

8.3 Colas y enrutamiento

Exchange `plataforma` (topic, durable). Clave de enrutamiento `evento.<Tipo>`. Cada consumidor tiene su cola durable propia:

Cola	Enlaces	Consumidor
auditoria	`#` (todo)	Auditoría
monitor	DeviceConnected, MitigationApplied	Monitor
deteccion	MetricSample	Detección
incidentes	AnomalyDetected, MitigationVerified, MitigationExpired, MAC_Moved	Incidentes
politicas	IncidentOpened, MitigationVerified, MitigationExpired, ElevationRequested, MAC_Moved	Políticas
iam	MAC_Moved	IAM
registro	DeviceConnected	Registro
consola	AnomalyDetected, IncidentOpened, DeviceRegistered, DeviceRevoked	Consola
inc.<id>	ciclo completo del incidente (además de las anteriores)	Incidentes; auto-borrada al cerrar; TTL de respaldo 24 h

8.4 Garantías de entrega

Parámetro	Valor	Para qué
Confirmación de publicación	activada (publisher confirms)	el publicador sabe que el broker recibió
Buffer local del publicador	10 000 mensajes, retroceso hasta 60 s	si RabbitMQ cae, no se detiene al servicio; el desborde se declara como **hueco**
Ack de consumo	manual, tras procesar	nada se descarta sin procesar
Reintentos	5, retroceso 1 s → 60 s	luego a `dlq.<cola>`
Ventana de duplicados	7 días	idempotencia
Buffer de reorden	100 mensajes por incidente, 10 s	hueco → se espera; si no llega, hueco declarado
Límite de cola	100 000 mensajes	DLX a la DLQ

8.5 Persistencia: una base por servicio

Base	Tablas principales	Retención
`bd_registro`	dispositivos(mac PK, tipo, titular, rol, estado, vigencia_hasta, responsable) · historial_dispositivo	indefinida
`bd_iam`	operadores(hash_bcrypt, totp_secreto cifrado, rol) · provision_totp · sesiones(sesion_id, usuario, rol, mac, switch_puerto, ip, idle/hard, estado, motivo_cierre) · tokens_derivados	sesiones: política de limpieza
`bd_incidentes`	incidentes(inc_id, tipo, objetivo, severidad, estado, ultima_secuencia) · eventos_incidente(secuencia, evento_id, payload_json) · mitigaciones(accion, alcance, tasa, ttl, cookie, estado, aprobacion)	indefinida
`bd_politicas`	perfiles · umbrales(destino, alfa, k_in, k_out, piso_pps, piso_bps, n_ventanas) · escalera(accion, automatica, condicion, ttl) · elevaciones	indefinida
`bd_monitor`	contadores(ts, switch, puerto_o_flujo, pps, bps) · lineas_base(destino, media, varianza) · features	**24 h crudo, 7 d agregado**
`bd_auditoria`	eventos_auditoria(ts, actor, accion, objetivo, detalle, evento_id, hash_previo) · exports	**90 días** (+ export JSONL)

- PostgreSQL 16, **una instancia, seis bases**, un usuario por base sin grants cruzados (hay que rechazar un acceso cruzado en pruebas).

- **El plano de datos manda**: ninguna base es la verdad de las reglas; tras una caída se reconcilia contra I2 (reglas vivas por cookie).
- **Auditoría**: export JSONL append-only con cadena SHA-256 (`hash = sha256(línea ‖ hash_anterior)`); el hash final abre el archivo siguiente. Una línea alterada rompe la cadena.
- **Respaldo**: `pg_dump` diario del volumen; drill de restauración programado.

9 R1 paso a paso — acceso según rol

Los 18 pasos de la Fig. 10: de «el dispositivo se conecta» a «la regla de sesión está en el ASIC del switch».

Referencia visual: Pres. Fig. 10 · lám. 7 (secuencia) y Pres. Fig. 11 · lám. 7 (escalera de perfiles). Aquí cada salto se explica con su protocolo, su mensaje y lo que pasa si falla. Actores: **Dispositivo** (host académico), **A1** (switch de ingreso), **ONOS** (maestro de A1), **Portal** .5, **IAM** .10, **FreeRADIUS** .23 + **LDAP** .24, **RabbitMQ** .21, **Políticas** .15.

9.1 Fase 1 — pasos 1–4: el dispositivo aún no es nadie (BASE)

#	De → A	Protocolo · puerto	Mensaje / acción	Resultado y por qué
1	Dispositivo → A1	Ethernet · DHCP UDP 67/68 · ARP	`DHCPDISCOVER` / ARP request	El dispositivo se anuncia. En el prototipo los hosts académicos tienen IP fija (reproducibilidad); la regla p10 igualmente deja pasar DHCP.
2	A1 → ONOS	OpenFlow 1.3 · TCP 6653	`PACKET_IN` (table-miss) con la trama y `in_port`	Primer paquete del dispositivo; sube al **maestro** de A1. ONOS aprende MAC, IP y puerto → `DeviceConnected` (a Registro, Monitor, Auditoría).
3	ONOS → A1	OpenFlow 1.3	`FLOW_MOD` ×6 esqueleto BASE (p150, p120, p110, p100, p10) + `BARRIER`	Queda instalado el perfil BASE: portal, par MAC/IP, anti-spoofing, DROP a gestión, DHCP/DNS. Cookies de tipo esqueleto, permanentes.
4	Navegador → Portal	HTTP 80 → HTTPS 443	Redirección al portal (regla p150) → `GET /login`	El único destino alcanzable fuera de DHCP/DNS es el portal 10.0.0.5. Redirección por grupo ALL o PACKET_OUT (V4 decide).

QUÉ PUEDE SALIR MAL EN LA FASE 1

- Sin `buffer_id` utilizable (V7): se responde con `PACKET_OUT` sin buffer, un poco más de carga.
- ONOS caído: el dispositivo no recibe esqueleto y no llega al portal (**fallo cerrado**, coherente con P5). Con el clúster, otro nodo asume y el PACKET_IN siguiente se atiende.
- Anti-spoofing: un dispositivo que cambia de MAC conservando IP cae en p110 (DROP).

9.2 Fase 2 — pasos 5–11: se exige identidad para salir de BASE

#	De → A	Protocolo · puerto	Mensaje / acción	Resultado y por qué
5	Navegador → Portal → IAM	HTTPS 443 → REST/JSON (I3)	`POST /login {usuario, clave [, TOTP], poblacion}`	El portal solo reenvía. Selector explícito: comunidad u operación.
6	IAM → FreeRADIUS → LDAP	RADIUS UDP 1812 → LDAP TCP 389 (I4)	`Access-Request` → bind/search `uid`	Solo comunidad. 3 s por intento, 2 reintentos; si agota, el login falla (síncrono).
7	FreeRADIUS → IAM	RADIUS UDP 1812	`Access-Accept` con `Class=perfil` (o Reject)	IAM traduce identidad + atributos. El IAM **no guarda** la clave de la comunidad.

8	IAM (interno)	—	Operador: verifica bcrypt + **TOTP** (RFC 6238, ±1)	Sin variante sin TOTP. Código incorrecto ⇒ rechazo, auditado.
9	IAM → ONOS	REST/HTTPS · TCP 8181 (I2)	`GET /api/v1/asociaciones?ip=…`	ONOS responde la tupla vigente `(MAC, switch:puerto, IP)`. La sesión se **liga** a ella.
10	IAM → RabbitMQ	AMQP 0-9-1 · TCP 5672 (I5)	Publica `SessionOpened` (identidad, perfil, dispositivo)	Evento **informa**; todavía no hay regla. Persistida en `bd_iam.sesiones` (estado activa).
11	RabbitMQ → Políticas	AMQP (cola durable `politicas`)	Entrega `SessionOpened`	Si Políticas estaba caída, procesa al volver (cola durable).

```
-- Paso 10: evento publicado por el IAM
{ "tipo":"SessionOpened", "emisor":"iam",
  "correlacion":{"sesion":"SES-000123","dispositivo":"52:54:00:01:01"},
  "payload":{"identidad":"jperez","perfil_o_rol":"ACADEMICO",
             "dispositivo":{"mac":"52:54:00:01:01","switch":"SA1","puerto":"p3","ip":"10.1.0.10"}} }
```

9.3 Fase 3 — pasos 12–18: la decisión baja a la red y se confirma

#	De → A	Protocolo · puerto	Mensaje / acción	Resultado y por qué
12	Políticas (interno)	—	ABAC: rol ∧ contexto ∧ vigencia ∧ estado de seguridad	Calcula el **perfil efectivo**. Si el dispositivo está bajo mitigación, la política lo considera.
13	Políticas → ONOS	REST/HTTPS · TCP 8181 (I2, síncrono)	`POST /api/v1/ordenes {APLICAR_PERFIL, dispositivo, vigencia, cookie SES-…}`	Única vía hacia la red. Idempotente por cookie.
14	ONOS → A1	OpenFlow 1.3 · TCP 6653	`FLOW_MOD` p200 (`cookie` SES, `idle_timeout` 1800, `hard` 43200, flag SEND_FLOW_REM) + `BARRIER_REQUEST`	Traduce el perfil a reglas de alcance (match por MAC/IP origen, destinos académicos + Internet). ONOS elige prioridades y matches; Políticas no.
15	A1 → ONOS	OpenFlow 1.3	`BARRIER_REPLY`	La regla está en hardware (no solo aceptada).
16	ONOS → Políticas	HTTP 201 · JSON	`{estado:"APLICADA", reglas:[…]}`	Primer nivel de confirmación.
17	Políticas → RabbitMQ	AMQP (I5)	Eco: perfil aplicado → Auditoría, Consola	Trazabilidad. El IAM marca la sesión efectiva.
18	Portal → Navegador	HTTPS 443	`200` + cookie opaca (`HttpOnly; Secure; SameSite=Lax`)	El usuario ve la sesión activa. Desde ahora el tráfico del dispositivo coincide p200 en A1 a velocidad de línea.

```
-- Paso 14: lo que ONOS instala en A1 (ilustrativo)
OFPT_FLOW_MOD command=ADD table=0 priority=200 cookie=<SES-000123>
   idle_timeout=1800 hard_timeout=43200 flags=SEND_FLOW_REM
   match: in_port=p3 eth_src=52:54:00:01:01 ipv4_src=10.1.0.10
   instr: apply_actions → (caminos p20 deciden la salida)
OFPT_BARRIER_REQUEST  →  OFPT_BARRIER_REPLY   ⇒ 201 APLICADA
```

PUNTO DE IMPLEMENTACIÓN ABIERTO

La Fig. 10 muestra el `200` al navegador *después* del eco (paso 17). Hay que decidir cómo espera el portal: (a) long-poll del IAM hasta ver la sesión efectiva, o (b) respuesta 200 inmediata con pantalla «aplicando» y polling por `GET /sesion`. Cualquiera cumple la lámina; recomendamos (b) para no mantener conexiones colgadas.

9.4 Salida de la sesión

Causa	Quién lo detecta	Qué ocurre
`POST /logout`	Portal → IAM	`RETIRAR` por cookie + `SessionClosed` inmediatos
`idle_timeout` (30 min)	switch (`FLOW_REMOVED`)	ONOS avisa → IAM cierra sesión, `Stop` RADIUS
`hard_timeout` (12 h)	switch	igual
`MAC_Moved`	Monitor → IAM	sesión invalidada, puerto vuelve a BASE
cookie desde otra IP	IAM en cada petición	rechazo + invalidación

9.5 Métricas y pruebas de R1

- **Métricas**: latencia login→regla (pasos 5→15), tasa de éxito/fallo, logins/s sostenidos.
- **Pruebas**: CU-01 (conexión y login), CU-02 (operador con TOTP), esqueleto instalado con 6 entradas y sus prioridades, TOTP incorrecto, cookie desde otra IP, sesión que vence al DROP a servicios y nunca abre gestión.

10 R2 paso a paso — recursos privilegiados

Los 16 pasos de la Fig. 12: pedir, aprobar, ordenar, instalar, usar y expirar una elevación.

Referencias: Pres. Fig. 12 · lám. 8 (secuencia), Pres. Fig. 13 · lám. 9 (clasificación), Pres. Fig. 14 · lám. 9 (pipeline), Pres. Fig. 15 · lám. 9 (reglas) y Pres. Fig. 16 · lám. 9 (capacidad).

10.1 Clasificación de los recursos

Clase	Recurso	Cómo se accede	Controles
General	servidor .10 (10.2.0.10)	perfil ACADÉMICO basta	p200 de sesión; DROP en ingreso si no hay sesión
Privilegiado	servidor .11 (10.2.0.11)	**elevación** + TTL + aprobación humana	p200 ELV-… con match por `ipv4_dst`; hard_timeout
Crítico	servidor .12 (BD institucional) · ONOS ×3 · IdP · switches · logs	solo VLAN 20 u OOB; operadores con rol	p100 DROP a gestión permanente; la infraestructura de la solución es crítica

En v2 los tres servidores viven en la zona C1/C2. Eso da un **segundo punto de control**: C1/C2 repiten el DROP a gestión y solo aceptan hacia VLAN 30 lo que llega por los caminos instalados.

10.2 Los 16 pasos

Solicitud — la elevación se pide, no se toma (1–5)

#	De → A	Protocolo · puerto	Mensaje / acción	Resultado y por qué
1	Usuario → Portal	HTTPS 443	`POST /elevacion {recurso:10.2.0.11, motivo}`	Requiere sesión ACADÉMICA vigente (cookie válida y ligada).
2	Portal → IAM	REST/HTTPS (I3)	solicitud de elevación	El IAM valida sesión y ligadura.
3	IAM → Registro	REST/HTTPS	¿MAC registrada y rol elegible?	El Registro responde si el dispositivo/rol puede aspirar a ese recurso (P7: la MAC es atributo).
4	IAM → RabbitMQ	AMQP 5672 (I5)	`ElevationRequested {solicitante, perfil_pedido, alcance, duracion}`	Evento a Políticas, Consola, Auditoría. Estado «pendiente».
5	RabbitMQ → Consola	AMQP	alerta en vivo	El administrador ve la solicitud.

Aprobación — un humano decide (6–7)

#	De → A	Protocolo · puerto	Mensaje / acción	Resultado y por qué
6	Admin → Consola → Políticas	REST/HTTPS (I7)	`POST /elevaciones/{id}/aprobar` (sesión admin + TOTP)	El aprobador queda en auditoría como `otorgante`.
7	Políticas (interno)	—	ABAC: rol ∧ recurso ∧ horario ∧ estado de seguridad ✓	Último filtro automático (p. ej. dispositivo bajo mitigación no se eleva).

Instalación — la decisión baja a la red (8–12)

#	De → A	Protocolo · puerto	Mensaje / acción	Resultado y por qué

8	Políticas → ONOS	REST/HTTPS · TCP 8181 (I2)	`POST /api/v1/ordenes {APLICAR_PERFIL, perfil:ELEVADO, alcance:10.2.0.11, hard_timeout_s:3600, cookie:ELV-0042}`	Orden síncrona.
9	ONOS → A1	OpenFlow 1.3	`FLOW_MOD` p200 `ipv4_src=10.1.0.10 → ipv4_dst=10.2.0.11`, `hard_timeout=3600`, SEND_FLOW_REM	Regla acotada: solo ese par origen/destino. Ver ejemplo en cap. 10.3.
10	A1 → ONOS	OpenFlow 1.3	`BARRIER_REPLY`	Regla en hardware.
11	ONOS → Políticas	HTTP 201	`APLICADA`	Confirmación.
12	Políticas → RabbitMQ	AMQP	eco `ElevationGranted` → Auditoría, Consola	Trazabilidad con cookie `ELV-0042`.

Uso y expiración — el permiso vive en el ASIC y caduca solo (13–16)

#	De → A	Protocolo · puerto	Mensaje / acción	Resultado y por qué
13	Usuario → srv .11	SSH TCP 22 / HTTPS 443	tráfico del host	Coincide p200 en A1 → D1/D2 → NC1/NC2 → C1 → srv. Ningún PACKET_IN (Fig. B-3).
14	A1 → ONOS	OpenFlow 1.3	`FLOW_REMOVED` (hard_timeout)	La regla expira sola a los 3600 s.
15	ONOS → Políticas	REST/HTTPS (callback app G3)	aviso de expiración	Políticas marca la elevación como expirada.
16	Políticas → RabbitMQ	AMQP	`ElevationExpired` → Auditoría	Cierra el ciclo Requested → Granted → Expired.

10.3 Reglas que realmente llegan al switch

```
// Paso 9 · elevación (lo que ONOS envía a A1)
OFPT_FLOW_MOD command=ADD table=0 priority=200
   cookie=0xE1000042 (ELV-0042) idle=0 hard_timeout=3600 flags=SEND_FLOW_REM
   match: in_port=p20 eth_src=52:54:00:01:01 eth_type=0x0800
          ipv4_src=10.1.0.10 ipv4_dst=10.2.0.11
   instr: apply_actions → output:p1 (hacia D1)
OFPT_BARRIER_REQUEST → OFPT_BARRIER_REPLY ⇒ 201 APLICADA
```

10.4 Camino denegado y defensa en profundidad

- Un académico **sin elevación** que intenta 10.2.0.11 no encuentra una p200 que lo permita: solo existe su sesión (alcance académico). El paquete cae en el esqueleto y se **descarta en A1**, antes de entrar al núcleo. No sube a ONOS.
- Si por error o ataque el tráfico llegara a C1/C2, ellos repiten el DROP a gestión y solo aceptan lo que cruza por caminos instalados (v2).
- Un dispositivo elevado sigue sujeto a mitigación: p1000 > p200 (P9).

10.5 Quién puede ordenar qué

Actor	Puede	No puede
Políticas	emitir órdenes I2	—
Consola	aprobar (I7) y consultar	hablar con ONOS o con los switches

IAM	identificar, crear sesión, publicar	ordenar a la red
Portal	recoger credenciales y solicitudes	decidir o validar

10.6 Pruebas y métricas

- CU-03 (acceso a servidor general) y CU-04 (elevación completa con TTL y expiración).
- Métricas: peticiones permitidas/denegadas y entradas TCAM por switch (llenado hasta TABLE_FULL).
- Verificar: tras la expiración, el mismo SSH ya no pasa (DROP en A1) y queda `ElevationExpired` en auditoría.

11 R4 paso a paso — DDoS y brute-force interno

El bucle cerrado observar → detectar → decidir → traducir → ejecutar → verificar, con el protocolo y los números de cada etapa.

Referencias: Pres. Fig. 17 · lám. 10 (bucle), Pres. Fig. 18 · lám. 10 (detección adaptativa), Pres. Fig. 19 · lám. 10 (escalera) y el cuadro de latencia. R4 vale 20 puntos y es el **diferenciador** del grupo. La tesis: **la regla instalada no es la mitigación verificada**.

11.1 Los 7 pasos del bucle

#	De → A	Protocolo · puerto	Mensaje / acción	Resultado y por qué
1	Monitor → ONOS → switches	REST GET (I8) → OF MULTIPART	`GET /api/v1/contadores` → `OFPMP_FLOW/PORT_STATS`	Cada 5 s (conjunto caliente) y 60 s (barrido). Una consulta por switch. Calcula deltas (pps, bps, flujos nuevos, fuentes distintas).
2	Monitor → Detección	AMQP 5672 (I5)	`MetricSample {destino, ventana, pps, bps, flujos_nuevos, fuentes_distintas}`	Materia prima. Persistida en `bd_monitor`; la EWMA se actualiza por destino.
3	Detección → Incidentes	AMQP	`AnomalyDetected {tipo, objetivo, origenes, tasas, linea_base, desviacion}`	Tras N=3 ventanas sostenidas por encima del umbral. Clasifica y propone severidad.
4	Incidentes → Políticas	AMQP	`IncidentOpened {INC-xxxx}`	Abre el incidente, cola `inc.INC-xxxx`, secuencia 1.
5	Políticas → ONOS	REST/HTTPS · TCP 8181 (I2)	`POST /api/v1/ordenes {APLICAR_MITIGACION, RATE_LIMIT, objetivo, origenes, ttl:300}`	Escalera según severidad/patrón. Eco al bus: `MitigationRequired`.
6	ONOS → switch de ingreso	OpenFlow 1.3 · TCP 6653	`METER_MOD` + `FLOW_MOD` p1000 (cookie INC) + `BARRIER`	El meter limita en el ASIC del switch donde **entra** el ataque. 201 APLICADA + `MitigationApplied`.
7	Monitor (verifica)	MULTIPART vía ONOS → AMQP	`MitigationVerified {tasas_antes, tasas_despues, cumple_objetivo}`	En la siguiente ventana de 5 s: ¿bajó el tráfico *real* hacia el destino? Si no cumple → ESCALADO.

11.2 Detección adaptativa (EWMA con piso)

Fórmulas (Monitor, por destino protegido y por indicador):

```
media_t    = media_{t-1} + α · (x_t − media_{t-1})                α = 0,2
varianza_t = varianza_{t-1} + α · ((x_t − media_t)² − varianza_{t-1})
σ          = √varianza
umbral     = media + max( k_in · σ , piso )        k_in = 4 ;  piso = 200 pps / 2 Mbps
alarma     = (x_t > umbral) durante N = 3 ventanas consecutivas de 5 s
salida     = (x_t < media + k_out · σ)             k_out = 2  (histéresis)
calentamiento: 30 min iniciales → solo alerta, no actúa
```

Mecanismo	Por qué existe
EWMA (adaptativa)	el tráfico del campus no es estacionario; un umbral fijo ciega en hora valle o dispara en hora punta
Piso absoluto	un servicio silencioso no debe disparar por pasar de 3 a 6 pps; el piso es estático por diseño
N ventanas	filtra picos breves legítimos (15 s de confirmación)

Histéresis 4σ/2σ	evita oscilación alrededor del umbral
Calentamiento 30 min	no hay línea base fiable al inicio; rigen los pisos y se alerta sin actuar

Ilustración del comportamiento en **Pres. Fig. 18 · lám. 10**. La escala √ del gráfico es solo visual.

11.3 Clasificación por patrón

Clase	Indicador	Regla inicial
Flood / DDoS	pps, bps hacia un destino	≥ 4σ y piso, 3 ventanas
Brute-force	solicitudes/s por origen	≥ 20/s sostenidas
Distribuido	fuentes distintas hacia un destino	≥ 10 fuentes en 60 s ⇒ se mitiga en **cada** ingreso (cada maestro ONOS instala en sus switches)
Scanning (R3)	destinos distintos por origen	> 20 destinos en 60 s

11.4 Escalera de mitigación

Peldaño	Efecto	Automatización	TTL
1 · RATE_LIMIT	meter a tasa objetivo (35 Mbps inicial en la presentación)	**Automático** ante anomalía confirmada	300 s
2 · BLOCK_SOURCE	DROP del origen	**Automático** si persiste pese al límite, o patrón inequívoco	corto
3 · ISOLATE_DEVICE	DROP total del dispositivo	**Automático solo ante detección determinista** (spoofing sostenido, MAC_Moved)	10 min
4 · QUARANTINE	bloqueo total de alto impacto	**Aprobación humana** (identificador de aprobación obligatorio)	el de la aprobación

PRECEDENCIA P9 / P12

Si una mitigación que preserva el tráfico legítimo no alcanza la métrica (el ataque baja pero no a la línea base), **prima P9 en la ejecución automática**: el sistema no sube la fuerza solo para «mejorar el número». P12 gobierna la evaluación: la brecha se mide, se registra y **escala a un humano**, que decide el siguiente peldaño. Regla única: se automatiza lo reversible y proporcional; lo total y de alto impacto pasa por una persona.

11.5 Ciclo de vida del incidente

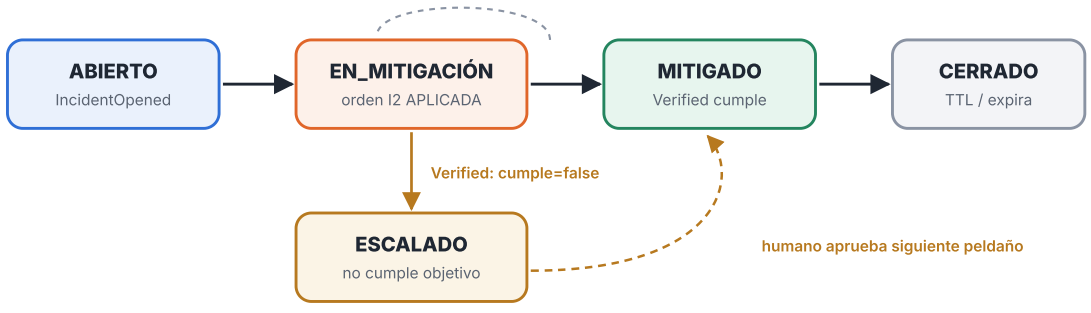


Fig. B-5 — Estados de un incidente (bd_incidentes). Cada evento se persiste con su secuencia *antes* de procesarlo: un reinicio del servicio retoma exactamente donde quedó.

11.6 Presupuesto de latencia



Fig. B-6 — Descomposición de los 185 ms (valores de diseño de la lámina 10). La detección domina el tiempo total (15 s por las 3 ventanas).

Etapa	Diseño (ms)	Notas
Evento AMQP	15	publicación + entrega con confirms
Política	25	ABAC + escalera
I2 REST/HTTPS	80	TLS + serialización + atravesar el clúster (puede incluir salto a otro maestro)
OpenFlow	45	FLOW_MOD + BARRIER
ASIC	20	programación de la regla en hardware
Suma	**185**	T_mitigate = 15 s + 0,185 s ≈ 15,2 s · vía rápida (N=1) ≈ 5,2 s

11.7 Por qué mitigar en el ingreso

- El ataque se corta **antes del núcleo**: protege también NC1/NC2, la zona C1/C2 y el canal de control in-band.
- No depende de ONOS en el momento del ataque: la regla ya vive en el switch (P2).
- Un ataque distribuido se mitiga en varios ingresos a la vez (A1, A2, A4). Cada nodo ONOS maestro instala en sus switches: demostración de concurrencia.

11.8 Reconciliación tras una caída

1. Al reiniciar, Incidentes lee sus mitigaciones registradas.
2. Consulta `GET /api/v1/reglas?cookie=INC-*` a ONOS (la verdad del plano de datos).
3. Toda mitigación registrada sin regla viva se marca **expirada** y se cierra (P10).
4. Los eventos faltantes se reponen desde la cola `inc.<id>` o se declaran como hueco.

11.9 Prueba y métricas

- **CU-07**: flood desde A2 (atacante interno) hacia 10.2.0.10 **con tráfico legítimo en paralelo**; se mide el tráfico legítimo antes/durante/después.
- **Métricas**: T_detect, T_mitigate, tráfico residual del atacante, falsos positivos, impacto sobre tráfico legítimo, carga de ONOS a 5/30/60 s (M1, M2).

12 R3 y R5 (diseño), fallos y pruebas

Lo que se diseña sin implementar, cómo falla cada componente y cómo se prueba todo.

12.1 R3 — Ataques encubiertos en la intranet (diseño)

Presentado en Pres. Fig. 20 · lám. 11. Principio: lo **determinista** se resuelve en el switch o por incoherencia; lo **estadístico** reutiliza la cadena de R4.

Ataque	Detección	Respuesta	Latencia
IP spoofing (IP legítima, MAC distinta)	regla p110 (match ip_src ∧ eth_src≠MAC)	DROP en el ASIC; contador alimenta Incidentes	sin latencia de decisión
MAC_Moved (misma MAC, otro puerto)	PORT_STATUS / PACKET_IN de otra ubicación → Monitor	IAM invalida la sesión; el puerto vuelve a BASE	sin umbral; inmediato
Scanning (> 20 destinos / 60 s)	features del Monitor → Detección	cadena R4: AnomalyDetected → escalera	N ventanas

La sesión ata identidad a ubicación: **moverse la rompe**. Pruebas: CU-05, CU-06.

12.2 R5 — Seguridad perimetral (diseño v2)

1. Un puerto SPAN en **NC1** y **NC2** replica el tráfico hacia la interfaz del sensor (sin IP, NIC-D).
2. **Suricata 7** (IDS, nunca inline) analiza con reglas propias (lista con vigencia y responsable por firma) y escribe `eve.json`.
3. El **Adaptador** traduce cada alerta a `AnomalyDetected` con `origen=perimetro` y lo publica por AMQP.
4. La cadena R4 completa decide: Incidentes → Políticas → I2 → **bloqueo en A4** (el switch de borde) por OpenFlow.

El espejo en el *núcleo* (no solo en el borde) ve también el tráfico este-oeste que cruza el núcleo Pres. Fig. 21 · lám. 11. Si Suricata cae, se degrada R5 pero **la red no se corta** (P8). Pendiente: listas negras dinámicas y filtrado DNS; sin feeds externos (RP-06).

> **COBERTURA DEL ESPEJO**
> Un SPAN en NC1/NC2 no ve el tráfico que no cruza el núcleo (p. ej. dos hosts bajo la misma distribución que se hablan dentro de D1). Es una limitación de visibilidad aceptada para el prototipo; se documenta en la defensa.

12.3 Matriz de fallos

Verde = redundancia desplegada y medible; ámbar = degradación explícita, nunca silenciosa Pres. Fig. 9 · lám. 6.

Falla	Comportamiento	Métrica	Prueba
Cae enlace A1–D1	PORT_STATUS → camino por D2 (fast-failover o recálculo)	tiempo de conmutación	RD1
Cae NC1	núcleo completo por NC2; C1 sigue por su enlace cruzado	tiempo de recuperación	RD2
Cae C1	servidores, plataforma y ONOS siguen por C2	pérdida de paquetes	RD3
Cae ONOS-1	switches piden MASTER a ONOS-2/3; reglas intactas	tiempo de re-maestría (M3)	RD4

Caen 2 nodos ONOS	sin quórum: reglas instaladas siguen; decisiones nuevas esperan	duración de degradación	—
Cae RabbitMQ	colas durables + DLQ; huecos declarados	tiempo de recuperación	—
Cae Detección	sin detecciones nuevas; mitigaciones vigentes siguen	duración de degradación	—
Cae Suricata	R5 pierde detección; red intacta	tiempo de recuperación	—
Cae PostgreSQL	servicios en not-ready; la red opera	tiempo de recuperación	—
Cae IAM	sin sesiones nuevas; vigentes siguen hasta timeout	—	—
Cae FreeRADIUS/LDAP	login de comunidad falla; operadores siguen	—	—

La regla común (P2): **la red no depende de la plataforma**. Un servicio caído detiene decisiones nuevas, no el reenvío ya instalado.

12.4 Verificaciones del plano de datos (V1–V7) — acta del laboratorio

ID	Qué verifica	Cómo	Si falla
V1	Modo OF 1.3 y capacidades (tablas, contadores por flujo/puerto, grupos); capacidad de TCAM	FEATURES_REPLY; llenado por lotes hasta TABLE_FULL	sin contadores por flujo no hay línea base: reevaluar pipeline
V2	Meters nativos	METER_FEATURES + METER_MOD real + METER_CONFIG	degradación: contención por reglas temporales
V3	Grupos (ALL, y fast-failover para caminos)	GROUP_MOD + GROUP_DESC	redirección por PACKET_OUT / reglas individuales
V4	Redirección al portal: grupo vs PACKET_OUT	experimento con tráfico pre-login	P12: quedarse con lo medido
V5	Expiración y retiro	idle_timeout corto + FLOW_REMOVED; retiro por cookie	sin FLOW_REMOVED: consulta periódica del inventario
V6	Rol de puerto y canal de control	PORT_STATUS al caer enlace; conexión rechazada desde otra interfaz	documentar la limitación
V7	Table-miss y buffer_id	todo paquete sin match llega como PACKET_IN	PACKET_OUT sin buffer

12.5 Métricas que el laboratorio debe cerrar

ID	Métrica	Rango / objetivo
M1	T_detect · T_mitigate · residual · falsos positivos	≤ 20 s peor caso lento
M2	Carga de ONOS con sondeo a 5/30/60 s	RNF-03
M3 (v2)	Tiempo de re-maestría del clúster	medir con RD4
M4 (v2)	Latencia de I2 (REST) por nodo	comparar nodos
RNF-01	Ligadura de sesión (cookie desde otra IP)	rechazo 100 %
RNF-05	Capacidad de tabla	entradas hasta TABLE_FULL
RNF-09	Desfase de reloj entre nodos	máximo observado (NTP)

12.6 Casos de uso de prueba

Caso	Qué ejerce	Req.
CU-01	conexión + login académico (cap. 9)	R1

CU-02	login de operador con TOTP	R1
CU-03	acceso a servidor general y denegación a gestión	R2
CU-04	elevación con TTL, expiración y auditoría (cap. 10)	R2
CU-05	IP spoofing (p110)	R3
CU-06	MAC_Moved y scanning	R3
CU-07	DDoS/brute-force con tráfico legítimo en paralelo (cap. 11)	R4
CU-08	alerta perimetral → bloqueo en A4	R5
RD1–RD4	redundancia: enlace, NC1, C1, ONOS-1	HA v2

13 Configuración, arranque, decisiones y abiertos

Los parámetros en un solo lugar, el orden en que se levanta el sistema, el registro de decisiones y lo que aún no está cerrado.

13.1 Parámetros de comportamiento (valores iniciales)

Se ajustan por la vía administrativa (I7) y cada cambio queda en auditoría con su autor. Los definitivos los calibra la medición (P12).

Área	Parámetro	Valor inicial
Detección	α · ventana · N · k_in/k_out · piso · calentamiento	0,2 · 5 s · 3 · 4σ/2σ · 200 pps/2 Mbps · 30 min
Clasificación	flood · distribuido · brute-force · scanning	≥4σ+piso · ≥10 fuentes/60 s · ≥20 req/s · >20 destinos/60 s
Observación (I8)	caliente · barrido · timeout de consulta	5 s · 60 s · 2 s
Mitigación	RATE_LIMIT TTL · ISOLATE TTL · tasa inicial	300 s · 10 min · 35 Mbps (o 1000 pps según V2)
Sesión ACADÉMICO	idle · hard	30 min · 12 h
Sesión OPERADOR	idle · hard	15 min · 8 h
Elevación	TTL	1 h (3600 s)
TOTP	estándar · ventana	RFC 6238 · ±1
RADIUS	timeout · reintentos · Interim	3 s · 2 · 300 s
OpenFlow	ECHO · pérdida declarada	5 s · 3 latidos
Bus	buffer · reintentos · duplicados	10 000 · 5 (1→60 s) · 7 días
Retenciones	auditoría · contadores	90 d · 24 h crudo / 7 d agregado

13.2 Configuración por componente

Componente	Configuración clave
ONOS (cada nodo)	OF 1.3; apps: openflow, rest, app G3; Atomix con los 3 miembros; REST 8181 solo en gestión; clave de servicio por cliente; registro de cookies persistido
Switches Pica8	modo OpenFlow 1.3 dedicado; `ma1` con whitelist de los 3 controladores; ECHO 5 s; puerto espejo (NC1/NC2); buffer_id utilizable
RabbitMQ	vhost `plataforma`; exchange topic durable; usuario por servicio; políticas de colas con DLX
PostgreSQL	6 bases, usuario por base; pool por servicio; `pg_dump` diario
FreeRADIUS/LDAP	cliente único = IAM (secreto propio); bind de solo lectura; `ou=comunidad`
Suricata	modo IDS sobre interfaz espejo; `eve.json`; reglas propias con vigencia/responsable
Secretos	variables de entorno / almacén de secretos; nunca en el repositorio ni en texto claro compartido

13.3 Orden de arranque

El orden es un grafo de dependencias por capas: **tiempo → identidad → estado → política → observación**. La red **no** depende de los servicios: el grafo dice quién espera a quién para *arrancar*, no quién detiene a quién al

fallar.

1. **Tiempo** (chrony) — todos los nodos con hora común (RNF-09).
2. **Persistencia** (PostgreSQL) — seis bases, `pg_isready`.
3. **Intermediario** (RabbitMQ) — vhost, exchange, colas, políticas.
4. **Controlador y switches — v2: formar primero el clúster ONOS (3 nodos, Atomix con quórum)**, luego conectar switches (HELLO/FEATURES/ROLE) y reinstalar esqueletos. En diseño in-band: bootstrap escalonado desde C1 (Fig. B-1).
5. **Identidad institucional** (FreeRADIUS + OpenLDAP) — `radtest` de control.
6. **Servicios de la plataforma** — IAM, Registro, Incidentes, Políticas, Auditoría, Monitor, Detección, Portal, Consola; cada uno `/health` ready.
7. **Sensor y visualización** — Suricata confirma espejo; Grafana carga datasources.
8. **Verificación de arranque** — un PACKET_IN de prueba recorre aprendizaje → esqueleto → portal; un evento de prueba recorre el bus extremo a extremo. Recién entonces se declara operativo.

Apagado: inverso, con la auditoría exportando su retención antes de detenerse.

Producto	Versión fijada
ONOS	2.7 LTS
PostgreSQL	16
RabbitMQ	3.13
FreeRADIUS · OpenLDAP	3.2 · 2.6
Suricata	7
chrony	4.5
Grafana	11
Python (servicios propios)	3.12

13.4 Supuestos y diferencias a vigilar

Lista viva de lo que esta biblia asume o donde difiere de otra fuente. Cada punto debe cerrarse o aceptarse explícitamente.

#	Tema	Qué se asumió / dónde difiere	Acción sugerida
1	Clúster ONOS	Interpretamos «dos clúster» como **1 clúster de 3 nodos** (quórum Raft). Los 3 nodos corren en el mismo servidor físico.	Confirmar con el coach; documentar el límite del host único.
2	IP .30 / .40	ONOS-2 y ONOS-3 en 10.0.0.30 y 10.0.0.40 (la v1 solo define .20).	Registrar en el plan de direccionamiento.
3	Memoria 512 MB	Hosts académicos reducidos a 512 MB para que quepa el clúster (≈ 29/32 GB).	Probar la imagen mínima; plan B en cap. 4.4.
4	18 enlaces	Desglose de aristas reconstruido de la Fig. 5.	Actualizar `05_puertos.md` a v2.
5	Autenticación	Memoria: ADR «802.1X + RADIUS». Repo/presentación: portal cautivo + RADIUS; 802.1X para puertos sensibles.	Decidir y alinear lám. 7.
6	Cookies	El repo usa tipo en bits altos 0x01–0x04; la lámina 9 usa 0xE1…/0xC0… como ejemplo.	Fijar la convención en el registro de ONOS.
7	RATE_LIMIT	Presentación: 35 Mbps (KBPS). Repo: 1000 pps (PKTPS).	Lo decide V2; unificar el ejemplo.
8	V3	Repo: V3 = grupos ALL. Presentación: V3 = fast-failover.	Ampliar V3 para cubrir ambos tipos de grupo.
9	Capacidad TCAM	≈ 2.000 entradas supuestas.	Medir con V1.

10	I2 en clúster	Cliente con lista de 3 nodos y reintento; no hay VIP.	Decidir si se pone un balanceador/VIP.
11	Espera del portal	Fig. 10 muestra 200 tras el eco; mecanismo (long-poll / polling) abierto.	Elegir (recomendado: polling).
12	Cobertura SPAN	El espejo en NC1/NC2 no ve tráfico que no cruza el núcleo.	Aceptar y documentar.

13.5 Registro de decisiones (ADR resumido)

Decisión	Motivo	Origen
OpenFlow 1.3 sobre Pica8/PicOS	hardware del laboratorio; contadores y meters	F-02
ONOS 2.7 como controlador	clúster nativo, REST, apps Java	F-01 / coach v2
Política en el switch de ingreso	menor latencia, protege el núcleo (P2, P9)	D / G-06
Control: diseño in-band, prototipo OOB	escala real vs medición sin riesgo	coach v2
Servidores sensibles en C1/C2 (núcleo)	menor superficie en el acceso, dual-homing	coach v2
Clúster ONOS de 3 nodos	quórum y tolerancia a 1 fallo	coach v2 (interpretación)
Broker AMQP + REST síncrono para órdenes	evento informa, comando ejecuta	D-08 / F-04
Una BD por servicio	aislamiento (Repository)	F-05
Escalera con QUARANTINE humano	lo total y de alto impacto pasa por una persona	F-06 / D-03
IDS fuera de camino (SPAN)	si cae, la red no se corta	F-07
Python 3.12 para los servicios propios	un lenguaje, ocho servicios, simplicidad	G-07

13.6 Glosario

Término	Significado
ABAC	control de acceso por atributos: rol, recurso, acción, contexto, vigencia, estado de seguridad
Atomix	almacén distribuido con Raft que usa ONOS para estado compartido
BARRIER	mensaje OpenFlow que confirma que los anteriores ya se aplicaron
Cookie (OF)	identificador de 64 bits de una regla; permite retirar/auditar por dueño
ECHO	latido del canal OpenFlow
EWMA	media móvil exponencial; base de la línea de normalidad
In-band / out-of-band	control por los mismos enlaces de datos / por una red aparte
LLDP	protocolo de descubrimiento de vecinos usado para la topología
Mastership	asignación de un nodo ONOS como único que escribe en un switch
Meter	mecanismo OpenFlow de limitación de tasa
PACKET_IN / PACKET_OUT	paquete del switch al controlador / del controlador al switch
Quórum	mayoría necesaria para decidir en Raft (2 de 3)
SPAN	puerto espejo que copia tráfico para el IDS
TCAM	memoria de hardware donde viven las reglas; capacidad finita
TOTP	contraseña de un solo uso basada en tiempo (RFC 6238)

13.7 Fuentes

- Repositorio JL24-d4h3/SDN-project (commit 9626b2b): docs/fases A–H, flows/00–05, access/00–02.
- Presentación parcial v2 (`G3_propuesta_parcial_v2.pdf`) — figuras 1–21.
- Observaciones del coach (reunión posterior a la v1) y enunciado del proyecto TEL354 2026-2.